

Automatic Safety Analysis of Smart Contracts

소순범

GIST 전기전자컴퓨터공학부

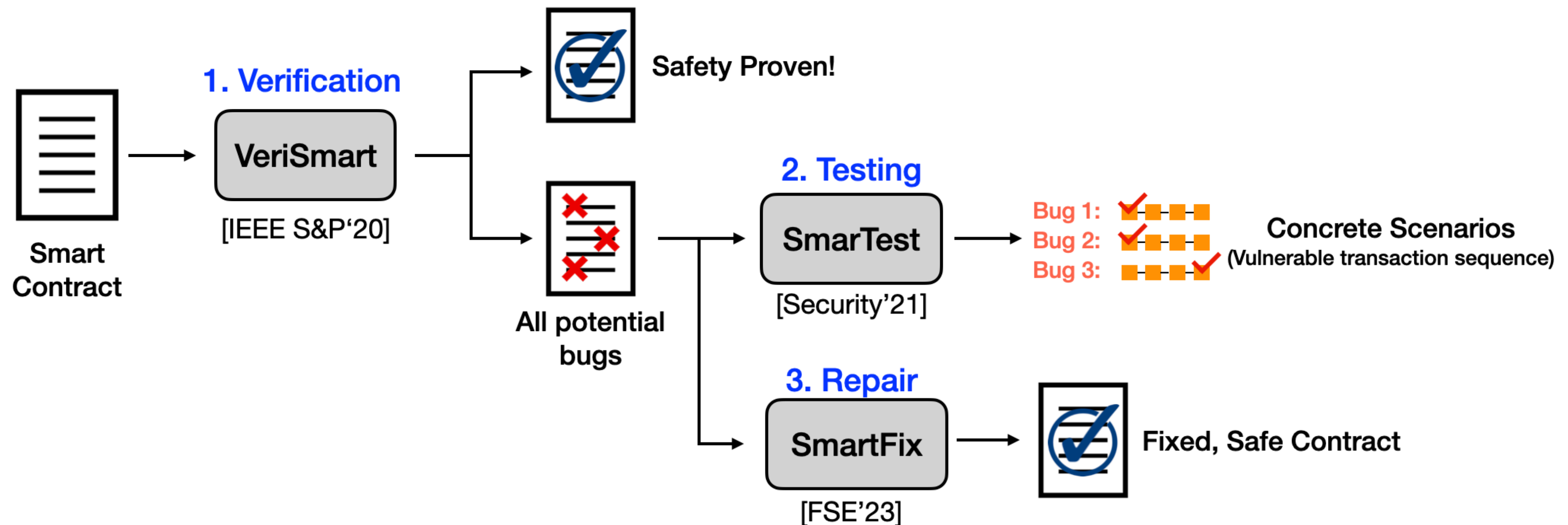
2024년 11월 15일

소프트웨어재난연구센터 세미나 (온라인)

This Talk:

Formal Methods for Smart Contract Security

1. **VeriSmart** (IEEE S&P 2020): formal verification of smart contracts
2. **SmarTest** (USENIX Security 2021): symbolic execution for finding vulnerabilities
3. **SmartFix** (FSE 2023): verification-based technique for fixing vulnerabilities



1. VeriSmart: Safety Verification

VeriSmart: A Highly Precise Safety Verifier for Ethereum Smart Contracts
IEEE Symposium on Security and Privacy 2020

Smart Contract

- Digital contract written in programming languages.

```
1  contract Netkoin {
2      mapping (address => uint) public balance;
3      uint public totalSupply;
4
5      constructor (uint initialSupply) {
6          totalSupply = initialSupply;
7          balance[msg.sender] = totalSupply;
8      }
9
10     function transfer (address to, uint value) public
11     returns (bool) {
12         require (balance[msg.sender] >= value);
13         balance[msg.sender] -= value;
14         balance[to] += value;
15         return true;
16     }
17
18     function burn(uint value) public returns (bool) {
19         require (balance[msg.sender] >= value);
20         balance[msg.sender] -= value;
21         totalSupply -= value;
22         return true;
23     }
24 }
```

Solidity Contract

Smart Contract

- Digital contract written in programming languages.

```
1  contract Netkoin {
2    mapping (address => uint) public balance;
3    uint public totalSupply;
4
5    constructor (uint initialSupply) {
6      totalSupply = initialSupply;
7      balance[msg.sender] = totalSupply;
8    }
9
10   function transfer (address to, uint value) public
11   returns (bool) {
12     require (balance[msg.sender] >= value);
13     balance[msg.sender] -= value;
14     balance[to] += value;
15     return true;
16   }
17
18   function burn(uint value) public returns (bool) {
19     require (balance[msg.sender] >= value);
20     balance[msg.sender] -= value;
21     totalSupply -= value;
22     return true;
23   }
24 }
```

State (global) variables

Constructor

Function

Function

Solidity Contract

Smart Contract

- Transaction execution = Function invocation

```
1  contract Netkoin {
2      mapping (address => uint) public balance;
3      uint public totalSupply;
4
5      constructor (uint initialSupply) {
6          totalSupply = initialSupply;
7          balance[msg.sender] = totalSupply;
8      }
9
10     function transfer (address to, uint value) public
11     returns (bool) {
12         require (balance[msg.sender] >= value);
13         balance[msg.sender] -= value;
14         balance[to] += value;
15         return true;
16     }
17
18     function burn(uint value) public returns (bool) {
19         require (balance[msg.sender] >= value);
20         balance[msg.sender] -= value;
21         totalSupply -= value;
22         return true;
23     }
24 }
```

Solidity Contract

balance[X] = 20,
balance[Y] = 0

transfer(Y, 5)
with X=msg.sender

balance[X] = 15,
balance[Y] = 5

X sends 5 tokens to Y.

Another Example: Auction Bidding

- A function that enables bidding in auction contracts.

```
1  function bid ( ) public returns (bool) {
2      require (now <= auctionEndTime);
3      require (msg.value > highestBid);
4
5      highestBidder = msg.sender;
6      highestBid = msg.value;
7
8      if (highestBid != 0) {
9          pendingReturns[highestBidder] += highestBid;
10     }
11 }
```

Another Example: Auction Bidding

- A function that enables bidding in auction contracts.

```
1  function bid ( ) public returns (bool) {
2      require (now <= auctionEndTime);
3      require (msg.value > highestBid);
4
5      highestBidder = msg.sender;
6      highestBid = msg.value;
7
8      if (highestBid != 0) {
9          pendingReturns[highestBidder] += highestBid;
10     }
11 }
```

Check if the bidding period is not over

Check if the current bid is the highest value

Another Example: Auction Bidding

- A function that enables bidding in auction contracts.

```
1  function bid ( ) public returns (bool) {
2      require (now <= auctionEndTime);
3      require (msg.value > highestBid);
4
5      highestBidder = msg.sender;
6      highestBid = msg.value;
7
8      if (highestBid != 0) {
9          pendingReturns[highestBidder] += highestBid;
10     }
11 }
```

Check if the bidding period is not over

Check if the current bid is the highest value

Set the current bidder as the highest bidder

Set the proposed bid as the highest bid

Another Example: Auction Bidding

- A function that enables bidding in auction contracts.

```
1  function bid ( ) public returns (bool) {  
2      require (now <= auctionEndTime);  
3      require (msg.value > highestBid);  
4  
5      highestBidder = msg.sender;  
6      highestBid = msg.value;  
7  
8      if (highestBid != 0) {  
9          pendingReturns[highestBidder] += highestBid;  
10     }  
11 }
```

Check if the bidding period is not over

Check if the current bid is the highest value

Set the current bidder as the highest bidder

Set the proposed bid as the highest bid

Record bids to send back Ethers to non-winning bidders

Increasing Popularity of Smart Contracts

- **No need for third parties' validation** when executing contracts.
 - since they run on the blockchain that provides **transparency** and **security**
- Becoming increasingly popular across diverse domains.



Importance of Safe Smart Contracts

- **Immutable** once deployed on blockchains.
- **Huge financial damage** once exploited.

(2016)

KLINT FINLEY 06.18.16 04:38 AM

A **\$50 Million** Hack Just Showed That the DAO Was All Too Human

(2021)

DIGITAL HEIST —

Really stupid “smart contract” bug let hackers steal **\$31 million** in digital coin

Company says it has contacted the hacker in an attempt to recover the funds. Good luck.

DAN GOODIN - 12/2/2021, 8:41 AM

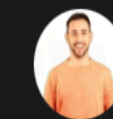
ETHEREUM > TECHNOLOGY

BatchOverflow Exploit Creates **Trillions** of Ethereum Tokens, Major Exchanges Halt ERC20 Deposits

Sam Town · April 25, 2018 at 10:38 pm UTC · 3 min read

(2018)

Gala Games became the victim of a **\$214 million** hack



by Jordan Avery — May 21, 2024 in News Reading Time: 3 mins read

(2024)

Goal: Ensuring safety before deployment

SmartMesh (CVE-2018-10376)

CVE-ID	
CVE-2018-10376	Learn more at National Vulnerability Database (NVD) • CVSS Severity Rating • Fix Information • Vulnerable Software Versions • SCAP Mappings • CPE Information
Description	
An integer overflow in the transferProxy function of a smart contract implementation for SmartMesh (aka SMT), an Ethereum ERC20 token, allows attackers to accomplish an unauthorized increase of digital assets via crafted _fee and _value parameters, as exploited in the wild in April 2018, aka the "proxyOverflow" issue.	

② ERC-20 Tokens Transferred: 2

► From [0xDF31A4...B34Eb46F](#) To [0xDF31A4...B34Eb46F](#) For

65,133,050,195,990,359,925,758,679,067,386,948,167,464,366,374,422,817,272,194.891004451135422463 ⌚ 6.29651431288701E+57

🚀 [SmartMesh...](#) (SMT...)

► From [0xDF31A4...B34Eb46F](#) To [0xd6a09B...e2e0651E](#) For

50,659,039,041,325,835,497,812,305,941,300,959,685,805,618,291,217,746,767,262.693003461994217473 ⌚ 4.89728891002323E+57

🚀 [SmartMesh...](#) (SMT...)

<https://etherscan.io/tx/0x1abab4c8db9a30e703114528e31dee129a3a758f7f8abc3b6494aad3d304e43f>

SmartMesh (CVE-2018-10376)

```
1  function transferProxy (address from, address to, uint value, uint fee)
2  public returns (bool) {
3      if (balance[from] < fee + value)
4          revert ();
5
6      if (balance[to] + value < balance[to] ||
7          balance[msg.sender] + fee < balance[msg.sender])
8          revert ();
9
10     balance[to] += value;
11     balance[msg.sender] += fee;
12     balance[from] -= value + fee;
13     return true;
14 }
```

SmartMesh (CVE-2018-10376)

```
1  function transferProxy (address from, address to, uint value, uint fee)
2  public returns (bool) {
3      if (balance[from] < fee + value)
4          revert ();
5
6      if (balance[to] + value < balance[to] ||
7          balance[msg.sender] + fee < balance[msg.sender])
8          revert ();
9
10     balance[to] += value;
11     balance[msg.sender] += fee;
12     balance[from] -= value + fee;
13     return true;
14 }
```

SmartMesh (CVE-2018-10376)

```
1  function transferProxy (address from, address to, uint value, uint fee)
2  public returns (bool) {
3      if (balance[from] < fee + value)
4          revert ();
5
6      if (balance[to] + value < balance[to] ||
7          balance[msg.sender] + fee < balance[msg.sender])
8          revert ();
9
10     balance[to] += value;
11     balance[msg.sender] += fee;
12     balance[from] -= value + fee;
13     return true;
14 }
```

Send money to the receiver (to)

SmartMesh (CVE-2018-10376)

```
1  function transferProxy (address from, address to, uint value, uint fee)
2  public returns (bool) {
3      if (balance[from] < fee + value)
4          revert ();
5
6      if (balance[to] + value < balance[to] ||
7          balance[msg.sender] + fee < balance[msg.sender])
8          revert ();
9
10     balance[to] += value;
11     balance[msg.sender] += fee;
12     balance[from] -= value + fee;
13     return true;
14 }
```

Send money to the receiver (to)

Pay fee to the proxy (msg.sender)

SmartMesh (CVE-2018-10376)

```
1  function transferProxy (address from, address to, uint value, uint fee)
2  public returns (bool) {
3      if (balance[from] < fee + value)
4          revert ();
5
6      if (balance[to] + value < balance[to] ||
7          balance[msg.sender] + fee < balance[msg.sender])
8          revert ();
9
10     balance[to] += value;
11     balance[msg.sender] += fee;
12     balance[from] -= value + fee;
13     return true;
14 }
```

Send money to the receiver (to)

Pay fee to the proxy (msg.sender)

Deduct money from the sender (from)

SmartMesh (CVE-2018-10376)

```
1  function transferProxy (address from, address to, uint value, uint fee)
2  public returns (bool) {
3      if (balance[from] < fee + value)
4          revert ();
5
6      if (balance[to] + value < balance[to] ||
7          balance[msg.sender] + fee < balance[msg.sender])
8          revert ();
9
10     balance[to] += value;
11     balance[msg.sender] += fee;
12     balance[from] -= value + fee;
13     return true;
14 }
```

To prevent underflows in the token sender's balance (L12)

Send money to the receiver (to)

Pay fee to the proxy (msg.sender)

Deduct money from the sender (from)

SmartMesh (CVE-2018-10376)

```
1  function transferProxy (address from, address to, uint value, uint fee)
2  public returns (bool) {
3      if (balance[from] < fee + value)
4          revert ();
5
6      if (balance[to] + value < balance[to] ||
7          balance[msg.sender] + fee < balance[msg.sender])
8          revert ();
9
10     balance[to] += value;
11     balance[msg.sender] += fee;
12     balance[from] -= value + fee;
13     return true;
14 }
```

To prevent underflows in the token sender's balance (L12)

To prevent overflows in the token recipient's balance (L10, L11)

Send money to the receiver (to)

Pay fee to the proxy (msg.sender)

Deduct money from the sender (from)

SmartMesh (CVE-2018-10376)

may overflow to 0!

```
1  function transferProxy (address from, address to, uint value, uint fee)
2  public returns (bool) {
3      if (balance[from] < fee + value)
4          revert ();
5
6      if (balance[to] + value < balance[to] ||
7          balance[msg.sender] + fee < balance[msg.sender])
8          revert ();
9
10     balance[to] += value;
11     balance[msg.sender] += fee;
12     balance[from] -= value + fee;
13     return true;
14 }
```

To prevent underflows in the token sender's balance (L12)

To prevent overflows in
the token recipient's balance
(L10, L11)

Send money to the receiver (to)

Pay fee to the proxy (msg.sender)

Deduct money from the sender (from)

SmartMesh (CVE-2018-10376)

256-bit unsigned integers in
hexadecimal numbers
(64 digits)

```
balance[from] = balance[to] = balance[msg.sender] = 0
```

```
value = 0x8fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
fee    = 0x7000000000000000000000000000000000000000000000000000000000000001
```

```
1  function transferProxy (address from, address to, uint value, uint fee)
2  public returns (bool) {
3      if (balance[from] < fee + value)
4          revert ();
5
6      if (balance[to] + value < balance[to] ||
7          balance[msg.sender] + fee < balance[msg.sender])
8          revert ();
9
10     balance[to] += value;
11     balance[msg.sender] += fee;
12     balance[from] -= value + fee;
13     return true;
14 }
```

SmartMesh (CVE-2018-10376)

256-bit unsigned integers in
hexadecimal numbers
(64 digits)

```
balance[from] = balance[to] = balance[msg.sender] = 0
```

```
value = 0x8fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
fee    = 0x7000000000000000000000000000000000000000000000000000000000000001
```

```
1  function transferProxy (address from, address to, uint value, uint fee)
2  public returns (bool) {
3      if (balance[from] < fee + value) false!
4          revert ();
5
6      if (balance[to] + value < balance[to] ||
7          balance[msg.sender] + fee < balance[msg.sender])
8          revert ();
9
10     balance[to] += value;
11     balance[msg.sender] += fee;
12     balance[from] -= value + fee;
13     return true;
14 }
```

0

SmartMesh (CVE-2018-10376)

256-bit unsigned integers in
hexadecimal numbers
(64 digits)

```
balance[from] = balance[to] = balance[msg.sender] = 0
```

```
value = 0x8fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
fee    = 0x7000000000000000000000000000000000000000000000000000000000000001
```

```
1  function transferProxy (address from, address to, uint value, uint fee)
2  public returns (bool) {
3      if (balance[from] < fee + value) false!
4          revert ();
5
6      if (balance[to] + value < balance[to] ||
7          balance[msg.sender] + fee < balance[msg.sender]) false!
8          revert ();
9
10     balance[to] += value;
11     balance[msg.sender] += fee;
12     balance[from] -= value + fee;
13     return true;
14 }
```

SmartMesh (CVE-2018-10376)

256-bit unsigned integers in
hexadecimal numbers
(64 digits)

```
balance[from] = balance[to] = balance[msg.sender] = 0
```

```
value = 0x8fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
fee    = 0x7000000000000000000000000000000000000000000000000000000000000001
```

```
1  function transferProxy (address from, address to, uint value, uint fee)
2  public returns (bool) {
3      if (balance[from] < fee + value) false!
4          revert ();
5
6      if (balance[to] + value < balance[to] ||
7          balance[msg.sender] + fee < balance[msg.sender]) false!
8          revert ();
9
10     balance[to] += value;
11     balance[msg.sender] += fee;
12     balance[from] -= value + fee;
13     return true;
14 }
```

Limitations of Existing Safety Analyzers

- Bug-finders could **fail to find critical bugs**.
 - E.g., Osiris [ACSAC '18], Oyente [CCS '16], Mythril, Manticore

```
1  function transferProxy (address from, address to, uint value, uint fee)
2  public returns (bool) {
3      if (balance[from] < fee + value)
4          revert ();
5
6      if (balance[to] + value < balance[to] ||
7          balance[msg.sender] + fee < balance[msg.sender])
8          revert ();
9
10     balance[to] += value;
11     balance[msg.sender] += fee;
12     balance[from] -= value + fee;
13     return true;
14 }
```

Only Osiris detects this vulnerability

CVE-2018-10376

Limitations of Existing Safety Analyzers

- Bug-finders could **fail to find critical bugs**.
 - E.g., Osiris [ACSAC '18], Oyente [CCS '16], Mythril, Manticore

```
1  function multipleTransfer (address[] to, uint value) public returns (bool) {
3      require(value * to.length > 0);
3      require(balance[msg.sender] >= value * to.length);
4      balance[msg.sender] -= value * to.length;
5
6      for (uint i =0; i < to.length; ++i) {
7          balance[to[i]] += value;
8      }
9      return true;
10 }
```

Despite the similarity,
Osiris now fails!

CVE-2018-14006

Limitations of Existing Safety Analyzers

- **Verifiers** typically suffer from **lots of false positives**.
 - E.g., Zeus [NDSS '18], SMTChecker [ISoLA '18]

```
1  contract Netkoin {
2      mapping (address => uint) public balance;
3      uint public totalSupply;
4
5      constructor (uint initialSupply) {
6          totalSupply = initialSupply;
7          balance[msg.sender] = totalSupply;
8      }
9
10     function transfer (address to, uint value) public
11     returns (bool) {
12         require (balance[msg.sender] >= value);
13         balance[msg.sender] -= value;
14         balance[to] += value;
15         return true;
16     }
17
18     function burn(uint value) public returns (bool) {
19         require (balance[msg.sender] >= value);
20         balance[msg.sender] -= value;
21         totalSupply -= value;
22         return true;
23     }
24 }
```

False alarm by Zeus, SMTChecker

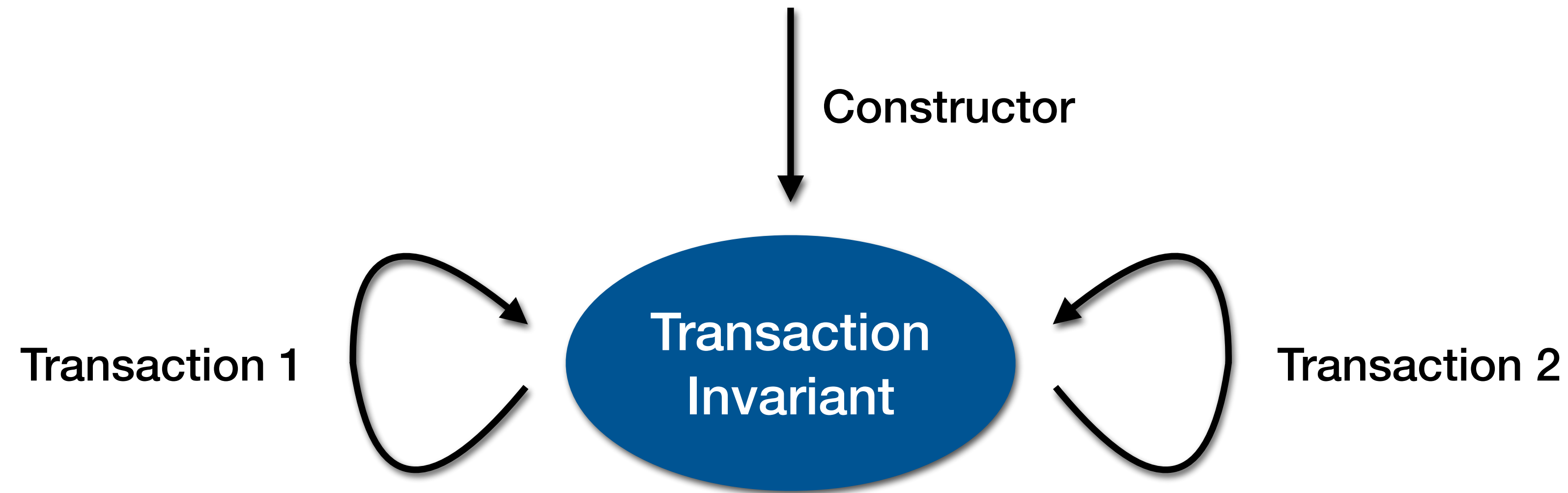
False alarm by Zeus, SMTChecker

Limitations of Existing Safety Analyzers

- **Bug-finders** could **fail to find critical vulnerabilities**.
 - Consider a subset of program paths.
 - E.g., Osiris [ACSAC '18], Oyente [CCS '16], Mythril, Manticore
- **Verifiers** **suffer from lots of false positives**.
 - Imprecise reasoning on global properties of smart contracts.
 - E.g., Zeus [NDSS '18], SMTChecker [ISoLA '18]

VeriSmart's Goal: overcome shortcomings of existing analyzers.

VeriSmart's Key Feature: Automatic Inference and Use of **Transaction Invariant**



- Global property that holds under arbitrary interleaving of transactions.
 1. Valid at the exit of the constructor.
 2. Validity preserved by executions of transactions.

VeriSmart's Key Feature:

Automatic Inference and Use of Transaction Invariant

```
1  contract Netkoin {
2    mapping (address => uint) public balance;
3    uint public totalSupply;
4
5    constructor (uint initialSupply) {
6      totalSupply = initialSupply;
7      balance[msg.sender] = totalSupply;
8    }
9
10   function transfer (address to, uint value) public
11   returns (bool) {
12     require (balance[msg.sender] >= value);
13     balance[msg.sender] -= value;
14     balance[to] += value;
15     return true;
16   }
17
18   function burn(uint value) public returns (bool) {
19     require (balance[msg.sender] >= value);
20     balance[msg.sender] -= value;
21     totalSupply -= value;
22     return true;
23   }
24 }
```

$$\text{totalSupply} = \sum_i \text{balance}[i]$$

False alarm by Zeus, SMTChecker

False alarm by Zeus, SMTChecker

VeriSmart's Key Feature:

Automatic Inference and Use of Transaction Invariant

```
1  contract Netkoin {
2    mapping (address => uint) public balance;
3    uint public totalSupply;
4
5    constructor (uint initialSupply) {
6      totalSupply = initialSupply;
7      balance[msg.sender] = totalSupply;
8    }
9
10   function transfer (address to, uint value) public
11   returns (bool) {
12     require (balance[msg.sender] >= value);
13     balance[msg.sender] -= value;
14     balance[to] += value;
15     return true;
16   }
17
18   function burn(uint value) public returns (bool) {
19     require (balance[msg.sender] >= value);
20     balance[msg.sender] -= value;
21     totalSupply -= value;
22     return true;
23   }
24 }
```

$\text{totalSupply} = \sum_i \text{balance}[i]$

$\text{totalSupply} = \sum_i \text{balance}[i]$

False alarm by Zeus, SMTChecker

False alarm by Zeus, SMTChecker

VeriSmart's Key Feature:

Automatic Inference and Use of Transaction Invariant

```
1  contract Netkoin {
2    mapping (address => uint) public balance;
3    uint public totalSupply;
4
5    constructor (uint initialSupply) {
6      totalSupply = initialSupply;
7      balance[msg.sender] = totalSupply;
8    }
9
10   function transfer (address to, uint value) public
11   returns (bool) {
12     require (balance[msg.sender] >= value);
13     balance[msg.sender] -= value;
14     balance[to] += value;
15     return true;
16   }
17
18   function burn(uint value) public returns (bool) {
19     require (balance[msg.sender] >= value);
20     balance[msg.sender] -= value;
21     totalSupply -= value;
22     return true;
23   }
24 }
```

$\text{totalSupply} = \sum_i \text{balance}[i]$

$\text{totalSupply} = \sum_i \text{balance}[i]$

$\text{totalSupply} = \sum_i \text{balance}[i]$

$\text{totalSupply} = \sum_i \text{balance}[i]$

False alarm by Zeus, SMTChecker

False alarm by Zeus, SMTChecker

VeriSmart's Key Feature:

Automatic Inference and Use of Transaction Invariant

```
1  contract Netkoin {
2    mapping (address => uint) public balance;
3    uint public totalSupply;
4
5    constructor (uint initialSupply) {
6      totalSupply = initialSupply;
7      balance[msg.sender] = totalSupply;
8    }
9
10   function transfer (address to, uint value) public
11   returns (bool) {
12     require (balance[msg.sender] >= value);
13     balance[msg.sender] -= value;
14     balance[to] += value;
15     return true;
16   }
17
18   function burn(uint value) public returns (bool) {
19     require (balance[msg.sender] >= value);
20     balance[msg.sender] -= value;
21     totalSupply -= value;
22     return true;
23   }
24 }
```

$\text{totalSupply} = \sum_i \text{balance}[i]$

$\text{totalSupply} = \sum_i \text{balance}[i]$

$\text{totalSupply} = \sum_i \text{balance}[i]$

$\text{totalSupply} = \sum_i \text{balance}[i]$

$\text{totalSupply} = \sum_i \text{balance}[i]$

$\text{totalSupply} = \sum_i \text{balance}[i]$

False alarm by Zeus, SMTChecker

False alarm by Zeus, SMTChecker

Precise Verification with Transaction Invariants

$\text{totalSupply} = \sum_i \text{balance}[i]$

$\text{totalSupply} = \sum_i \text{balance}[i]$

```
18  function burn(uint value) public returns (bool) {  
19      require (balance[msg.sender] >= value);  
20      balance[msg.sender] -= value;  
21      totalSupply -= value;  
22      return true;  
23  }  
24  }
```

False alarm by Zeus, SMTChecker

- To show: $\text{totalSupply} \geq \text{value}$ holds at line 21.

Precise Verification with Transaction Invariants

$\text{totalSupply} = \sum_i \text{balance}[i]$

$\text{totalSupply} = \sum_i \text{balance}[i]$

```
18  function burn(uint value) public returns (bool) {  
19      require (balance[msg.sender] >= value);  
20      balance[msg.sender] -= value;  
21      totalSupply -= value;  
22      return true;  
23  }  
24 }
```

False alarm by Zeus, SMTChecker

- To show: $\text{totalSupply} \geq \text{value}$ holds at line 21.
- Verification using the inferred invariant:

$\text{totalSupply} \geq \text{balance}[\text{msg.sender}]$

from “ $\text{totalSupply} = \sum_i \text{balance}[i]$ ”

Precise Verification with Transaction Invariants

$\text{totalSupply} = \sum_i \text{balance}[i]$

$\text{totalSupply} = \sum_i \text{balance}[i]$

```
18  function burn(uint value) public returns (bool) {  
19      require (balance[msg.sender] >= value);  
20      balance[msg.sender] -= value;  
21      totalSupply -= value;  
22      return true;  
23  }  
24 }
```

False alarm by Zeus, SMTChecker

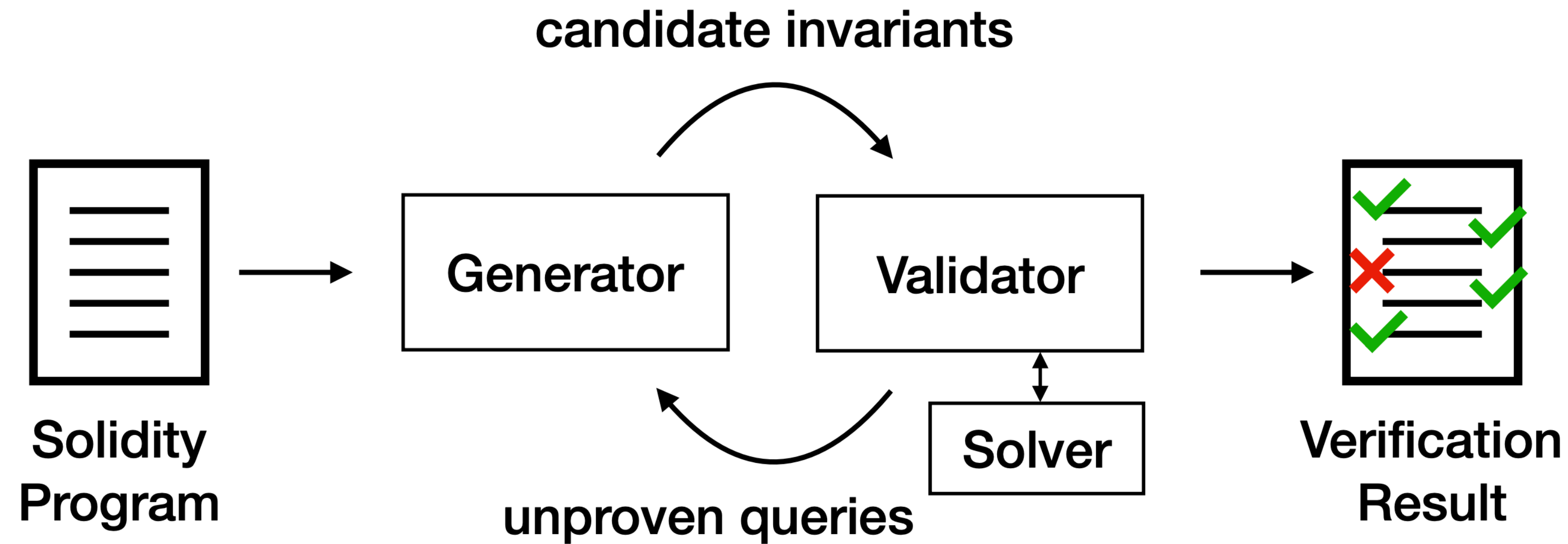
- To show: $\text{totalSupply} \geq \text{value}$ holds at line 21.
- Verification using the inferred invariant:

$\text{totalSupply} \geq \text{balance}[\text{msg.sender}] \geq \text{value}$

from " $\text{totalSupply} = \sum_i \text{balance}[i]$ "

line 19

VeriSmart Algorithm



- **Generator:** produce transaction/loop candidate invariants.
- **Validator:** validate the contract using candidate invariants.

Evaluation: vs. Bug-finders

	VeriSmart	Osiris [ACSAC '18]	Oyente [CCS '16]	Mythril (Consensys)	Manticore (Trail of Bits)
#Alarm	492	240	171	94	14
#False Positive	2	13	14	10	0
Recall (#valid CVE: 58)	100%	>> 70.7%	60.3%	19.0%	3.4%
Precision (#TP/ #Alarm)	99.6%	>> 94.6%	91.8%	89.4%	n/a (too low recall)

Results on 60 contracts with CVE vulnerabilities

Evaluation: vs. Verifiers

- Benchmark: 25 contracts where Zeus[NDSS'18] produced false positives.
 - VeriSmart verified **24 contracts** without false positives!

No.	LOC	#Q	VERISMA RT			SMTCHECKER [12]			ZEUS [11]
			#Alarm	#FP	Verified	#Alarm	#FP	Verified	
#1	42	3	0	0	✓	3	3	✗	✗
#2	78	2	1	0	✓	2	1	✗	✗
#3	75	7	2	0	✓	7	5	✗	✗
#4	70	7	0	0	✓	7	7	✗	✗
#5	103	8	0	0	✓	6	6	✗	✗
#6	141	5	2	0	✓	internal error			✗
#7	74	6	1	0	✓	6	5	✗	✗
#8	84	6	0	0	✓	4	4	✗	✗
#9	82	6	0	0	✓	6	6	✗	✗
#10	99	2	1	0	✓	internal error			✗
#11	171	15	9	0	✓	internal error			✗
#12	139	7	0	0	✓	internal error			✗
#13	139	7	0	0	✓	internal error			✗
#14	139	7	0	0	✓	internal error			✗
#15	139	7	0	0	✓	internal error			✗
#16	141	16	10	0	✓	internal error			✗
#17	153	5	0	0	✓	internal error			✗
#18	139	7	0	0	✓	internal error			✗
#19	113	4	0	0	✓	4	4	✗	✗
#20	40	3	0	0	✓	3	3	✗	✗
#21	59	3	0	0	✓	internal error			✗
#22	28	3	1	0	✓	1	0	✓	✗
#23	19	3	0	0	✓	3	3	✗	✗
#24	457	30	13	6	✗	internal error			✗
#25	17	3	0	0	✓	3	3	✗	✗
Total	2741	172	40	6	✓:24 ✗: 1	55	50	✓: 1 ✗: 12	✓: 0 ✗:25

 : No false positives
 : False positives exist

Evaluation: Importance of Transaction Invariants

only 8 contracts

- Without transaction invariants, VeriSmart verified ~~24 contracts~~ without false positives.
 - On 25 contracts from the Zeus [NDSS'18] dataset.
- VeriSmart found 6 incorrectly-reported CVE vulnerabilities.
 - By proving the safety at locations known to be vulnerable.

CVE ID	Name	#Incorrect Queries	#FP		
			OSIRIS	OYENTE	VERISmart
2018-13113	ETT	2	2	2	0
2018-13144	PDX	1	1	1	0
2018-13326	BTX	2	2	2	0
2018-13327	CCLAG	1	1	1	0

2. SmarTest: Symbolic Execution

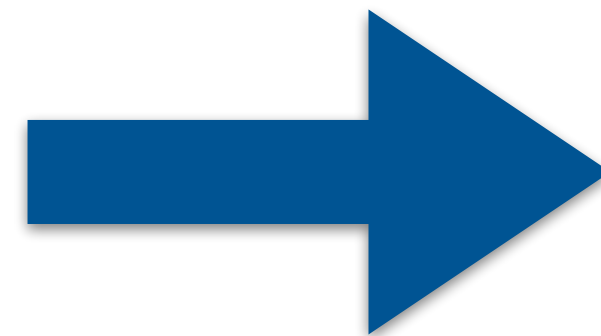
*SmarTest : Effectively Hunting Vulnerable Transaction Sequences in Smart Contracts
through Language Model-Guided Symbolic Execution*
USENIX Security 2021

SmarTest's Goal: Find Vulnerabilities with Concrete Scenarios

```
1  contract Example {
2      address owner;
3      mapping (address => uint) balance;
4      mapping (address => mapping (address => uint)) allowed;
5      uint totalSupply;
6
7      constructor () public {
8          owner = msg.sender;
9          totalSupply = 0;
10     }
11
12     function mintToken (address target, uint amount) {
13         require (owner == msg.sender);
14         balance[target] += amount;
15         totalSupply += amount;
16     }
17
18     function approve(address spender, uint value)
19     public returns (bool) {
20         allowed[msg.sender][spender] = value;
21         return true;
22     }
23
24     function burnFrom (address from, uint value)
25     public returns (bool) {
26         require (balance[from] >= value);
27         require (allowed[from][msg.sender] >= value);
28         balance[from] -= value;
29         allowed[from][msg.sender] -= value;
30         totalSupply -= value;
31         return true;
32     }
33 }
```

Underflow
(line 30)

Analyzer



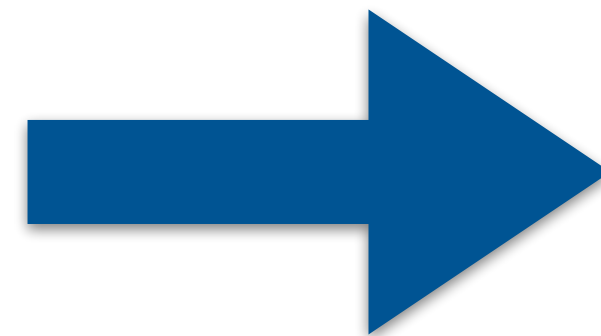
“There may be an
underflow vulnerability at L30.”

SmarTest's Goal: Find Vulnerabilities with Concrete Scenarios

```
1 contract Example {
2   address owner;
3   mapping (address => uint) balance;
4   mapping (address => mapping (address => uint)) allowed;
5   uint totalSupply;
6
7   constructor () public {
8     owner = msg.sender;
9     totalSupply = 0;
10  }
11
12  function mintToken (address target, uint amount) {
13    require (owner == msg.sender);
14    balance[target] += amount;
15    totalSupply += amount;
16  }
17
18  function approve(address spender, uint value)
19  public returns (bool) {
20    allowed[msg.sender][spender] = value;
21    return true;
22  }
23
24  function burnFrom (address from, uint value)
25  public returns (bool) {
26    require (balance[from] >= value);
27    require (allowed[from][msg.sender] >= value);
28    balance[from] -= value;
29    allowed[from][msg.sender] -= value;
30    totalSupply -= value;
31    return true;
32  }
33 }
```

Underflow
(line 30)

Analyzer



“There may be an
underflow vulnerability at L30.”

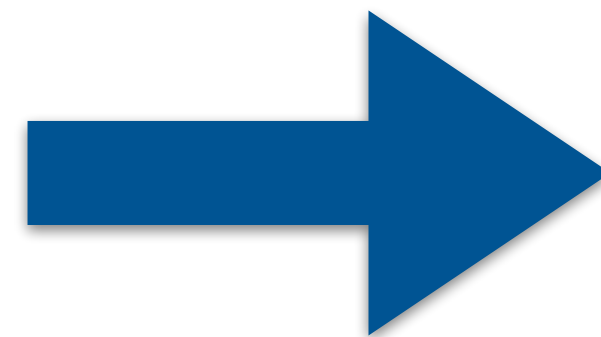
In what situations?

SmarTest's Goal: Find Vulnerabilities with Concrete Scenarios

```
1 contract Example {
2   address owner;
3   mapping (address => uint) balance;
4   mapping (address => mapping (address => uint)) allowed;
5   uint totalSupply;
6
7   constructor () public {
8     owner = msg.sender;
9     totalSupply = 0;
10  }
11
12  function mintToken (address target, uint amount) {
13    require (owner == msg.sender);
14    balance[target] += amount;
15    totalSupply += amount;
16  }
17
18  function approve(address spender, uint value)
19  public returns (bool) {
20    allowed[msg.sender][spender] = value;
21    return true;
22  }
23
24  function burnFrom (address from, uint value)
25  public returns (bool) {
26    require (balance[from] >= value);
27    require (allowed[from][msg.sender] >= value);
28    balance[from] -= value;
29    allowed[from][msg.sender] -= value;
30    totalSupply -= value;
31    return true;
32  }
33 }
```

Underflow
(line 30)

SmarTest



“vulnerable transaction sequence” (length 4)

“There ~~may be~~ ^{exists} an
underflow vulnerability at L30.”

mintToken (A, 1)

with msg.sender = owner

totalSupply = 1,
balance[A] = 1

mintToken (B, $2^{256} - 1$)

with msg.sender = owner

totalSupply = 0,
balance[B] = $2^{256} - 1$

approve (C, 1)

with msg.sender = B

allowed[B][C] = 1

burnFrom (B, 1)

with msg.sender = C

Overflow at line 15

Can pass line 26

Can pass line 27

Pass line 26, 27
Underflow at line 30

Basic Approach

- Symbolic execution in increasing order.

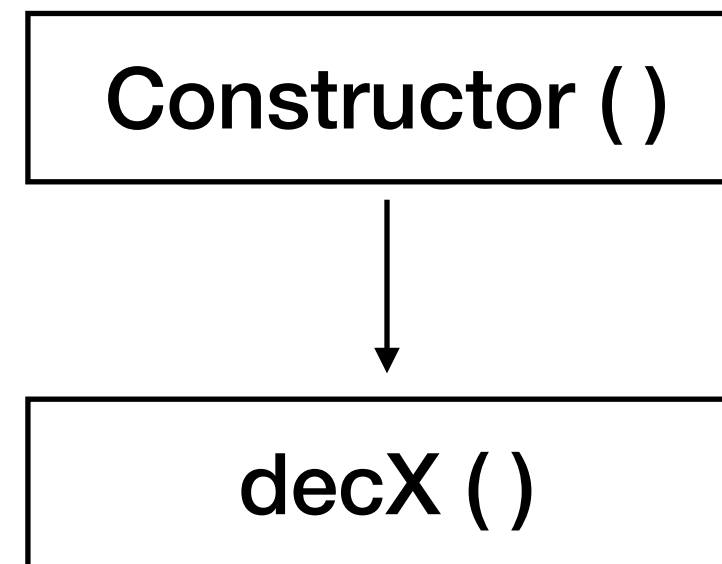
```
1  contract Example {
2    bool flag;
3    uint x;
4
5    constructor () public {
6      flag = false;
7      x = 100;
8    }
9
10   function setFlag () public {
11     flag = true;
12   }
13
14   function setX () public {
15     require (flag == true);
16     x = 0;
17   }
18
19   function decX () public {
20     require (flag == true);
21     require (x < 100);
22     x = x - 1;
23   }
24 }
```

Basic Approach

- Symbolic execution in increasing order.

```
1  contract Example {  
2    bool flag;  
3    uint x;  
4  
5    constructor () public {  
6      flag = false;  
7      x = 100;  
8    }  
9  
10   function setFlag () public {  
11     flag = true;  
12   }  
13  
14   function setX () public {  
15     require (flag == true);  
16     x = 0;  
17   }  
18  
19   function decX () public {  
20     require (flag == true);  
21     require (x < 100);  
22     x = x - 1;  
23   }  
24 }
```

Transaction Sequence



Verification Condition

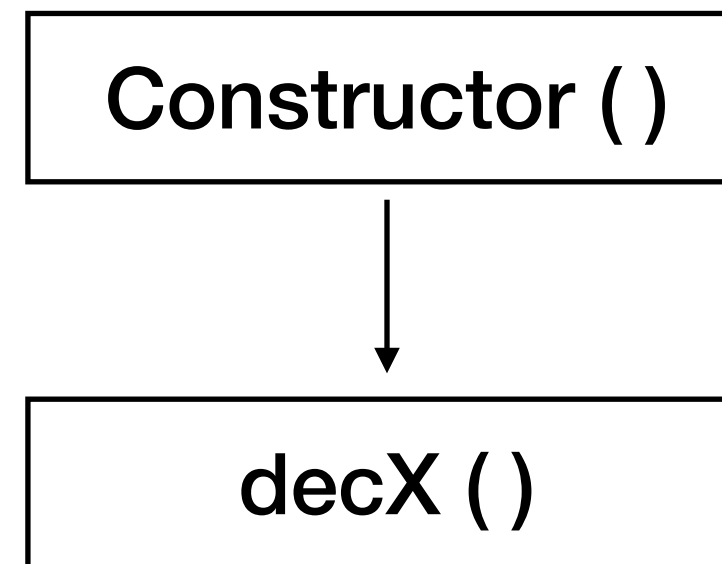
Goal:
Trigger underflow

Basic Approach

- Symbolic execution in increasing order.

```
1  contract Example {  
2    bool flag;  
3    uint x;  
4  
5    constructor () public {  
6      flag = false;  
7      x = 100;  
8    }  
9  
10   function setFlag () public {  
11     flag = true;  
12   }  
13  
14   function setX () public {  
15     require (flag == true);  
16     x = 0;  
17   }  
18  
19   function decX () public {  
20     require (flag == true);  
21     require (x < 100);  
22     x = x - 1;  
23   }  
24 }
```

Transaction Sequence



Verification Condition

$(flag = false) \wedge (x = 100)$

$\wedge (flag = true) \wedge (x < 100) \wedge \neg(x \geq 1)$

Negation of
underflow-safety condition

Goal:
Trigger underflow

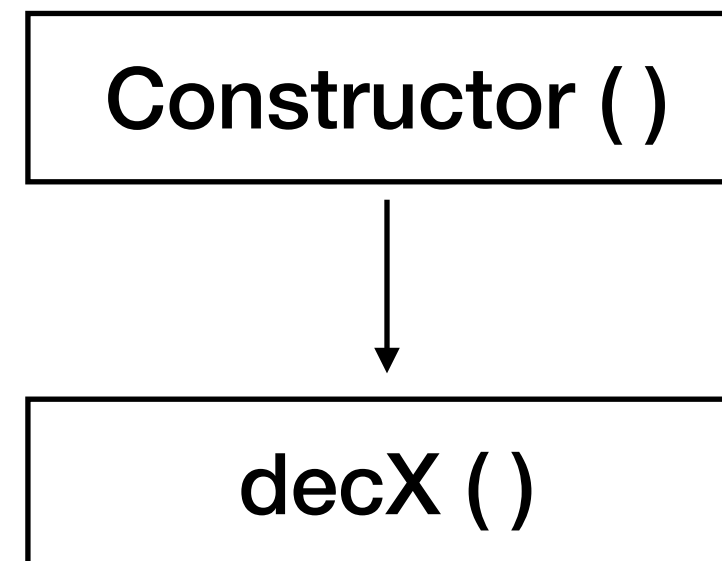
Basic Approach

- Symbolic execution in increasing order.

```
1  contract Example {
2    bool flag;
3    uint x;
4
5    constructor () public {
6      flag = false;
7      x = 100;
8    }
9
10   function setFlag () public {
11     flag = true;
12   }
13
14   function setX () public {
15     require (flag == true);
16     x = 0;
17   }
18
19   function decX () public {
20     require (flag == true);
21     require (x < 100);
22     x = x - 1;
23   }
24 }
```

Goal:
Trigger underflow

Transaction Sequence



Verification Condition

$(flag = false) \wedge (x = 100)$
Conflict!
 $\wedge (flag = true) \wedge (x < 100) \wedge \neg(x \geq 1)$
Conflict!

Negation of
underflow-safety condition

SMT Solver:
UNSAT!

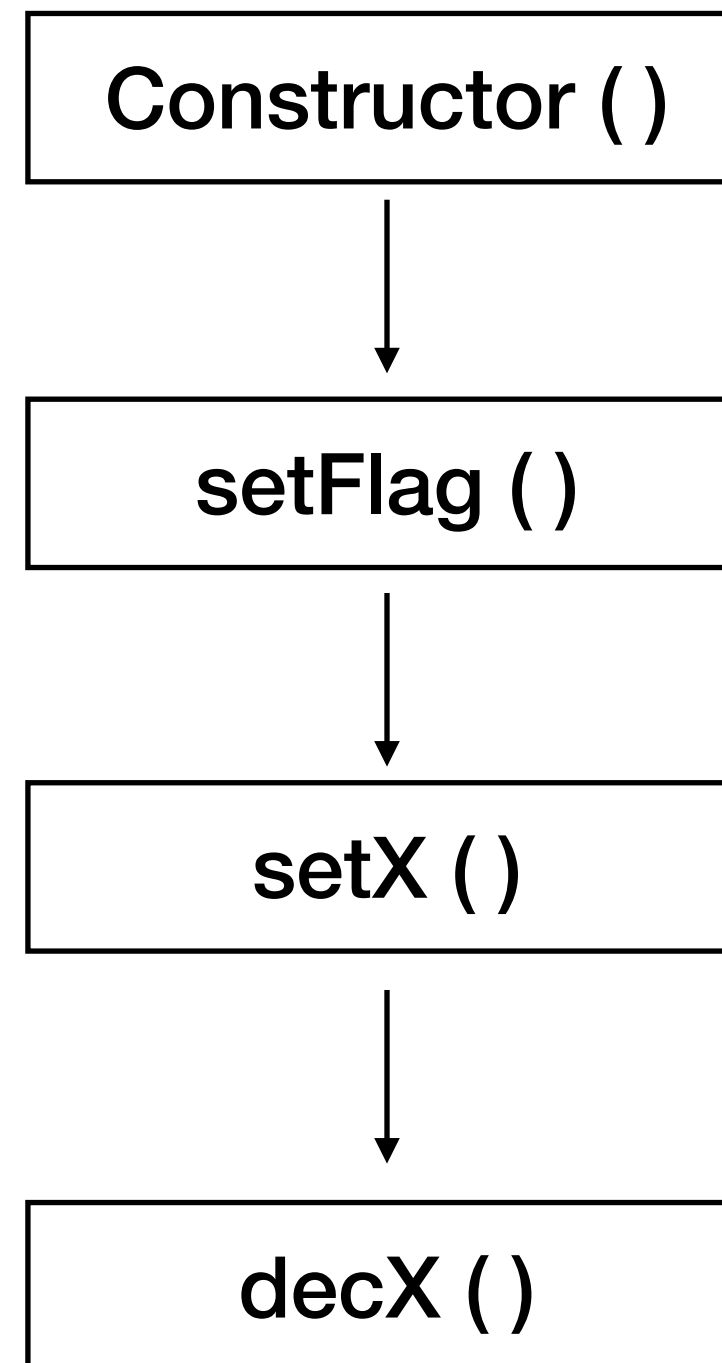
Basic Approach

- Symbolic execution in increasing order.

```
1  contract Example {  
2    bool flag;  
3    uint x;  
4  
5    constructor () public {  
6      flag = false;  
7      x = 100;  
8    }  
9  
10   function setFlag () public {  
11     flag = true;  
12   }  
13  
14   function setX () public {  
15     require (flag == true);  
16     x = 0;  
17   }  
18  
19   function decX () public {  
20     require (flag == true);  
21     require (x < 100);  
22     x = x - 1;  
23   }  
24 }
```

Goal:
Trigger underflow

Transaction Sequence



Verification Condition

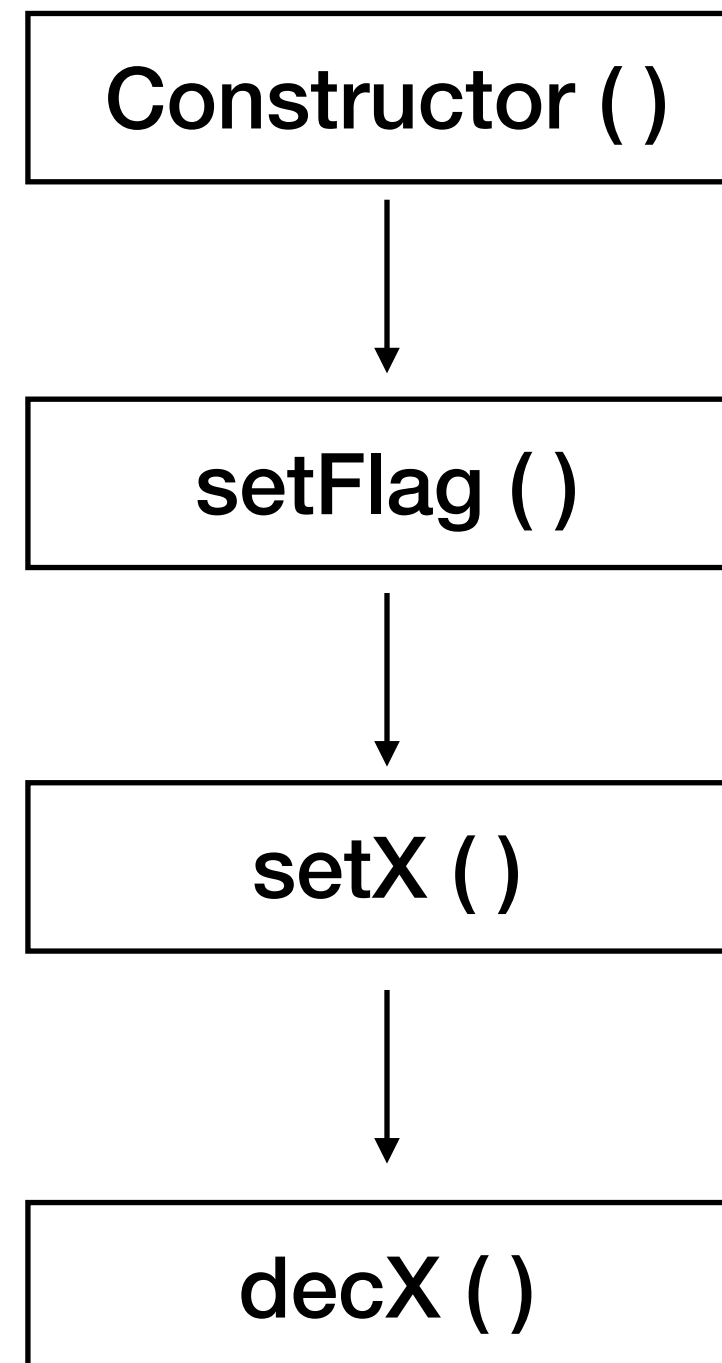
Basic Approach

- Symbolic execution in increasing order.

```
1  contract Example {
2    bool flag;
3    uint x;
4
5    constructor () public {
6      flag = false;
7      x = 100;
8    }
9
10   function setFlag () public {
11     flag = true;
12   }
13
14   function setX () public {
15     require (flag == true);
16     x = 0;
17   }
18
19   function decX () public {
20     require (flag == true);
21     require (x < 100);
22     x = x - 1;
23   }
24 }
```

Goal:
Trigger underflow

Transaction Sequence



Verification Condition

$(flag' = false) \wedge (x' = 100)$

$\wedge (flag = true)$

$\wedge (flag = true) \wedge (x = 0)$

$\wedge (flag = true) \wedge (x < 100) \wedge \neg(x \geq 1)$

Negation of
underflow-safety condition

Basic Approach

- Symbolic execution in increasing order.

Vulnerable Transaction
Sequence Found!

SMT Solver:
SAT!

Transaction Sequence

Verification Condition

$$(flag' = false) \wedge (x' = 100)$$

$$\wedge (flag = true)$$

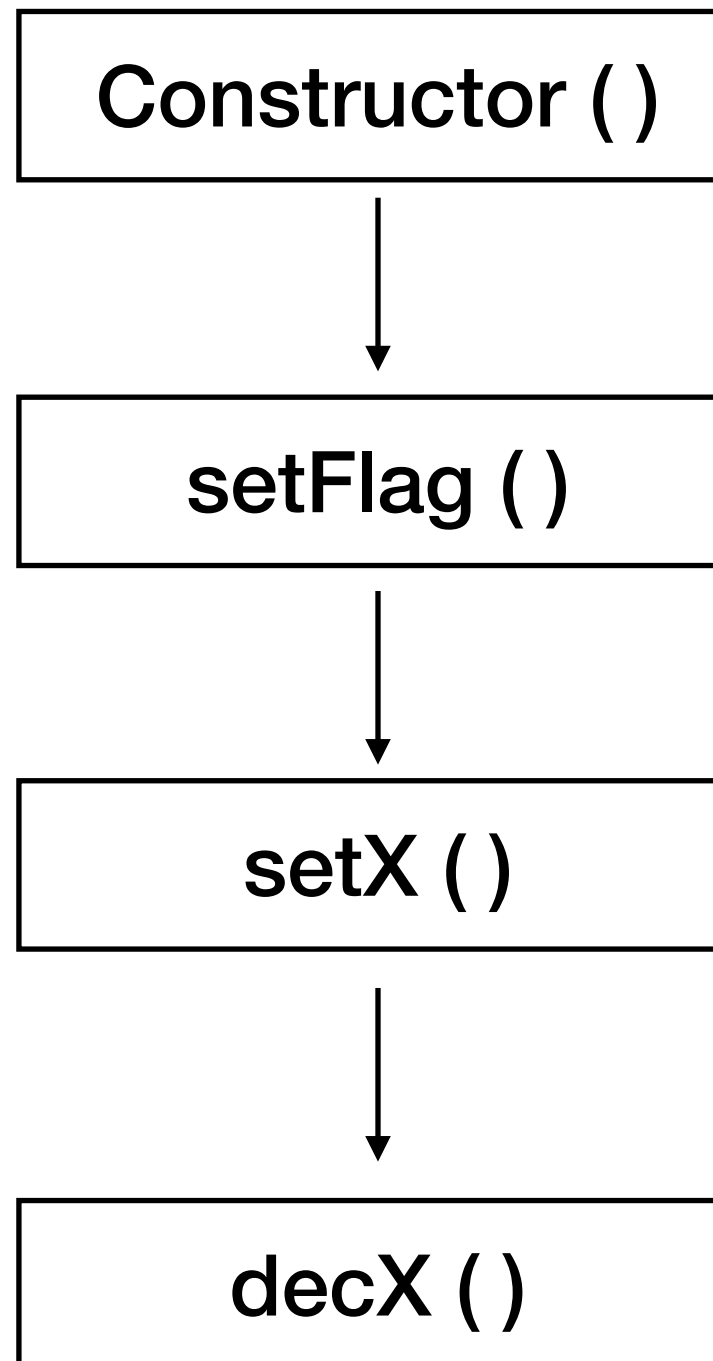
$$\wedge (flag = true) \wedge (x = 0)$$

$$\wedge (flag = true) \wedge (x < 100) \wedge \neg(x \geq 1)$$

Goal:
Trigger underflow

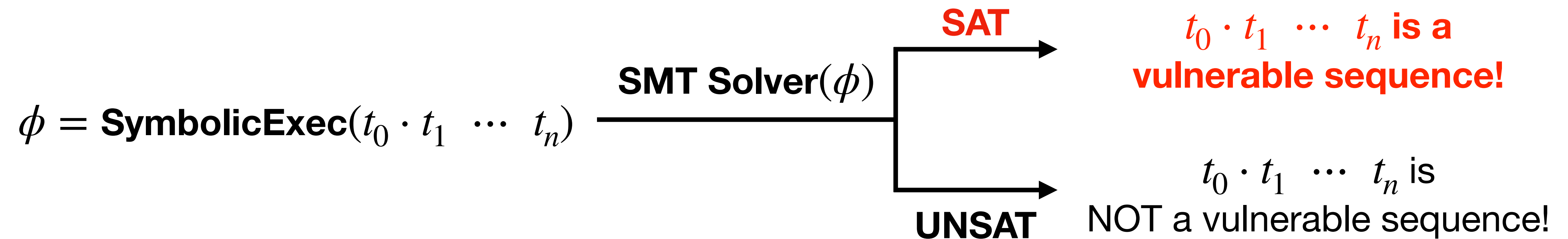
Negation of
underflow-safety condition

```
1  contract Example {
2    bool flag;
3    uint x;
4
5    constructor () public {
6      flag = false;
7      x = 100;
8    }
9
10   function setFlag () public {
11     flag = true;
12   }
13
14   function setX () public {
15     require (flag == true);
16     x = 0;
17   }
18
19   function decX () public {
20     require (flag == true);
21     require (x < 100);
22     x = x - 1;
23   }
24 }
```



Basic Approach: Symbolic Execution in Increasing Order

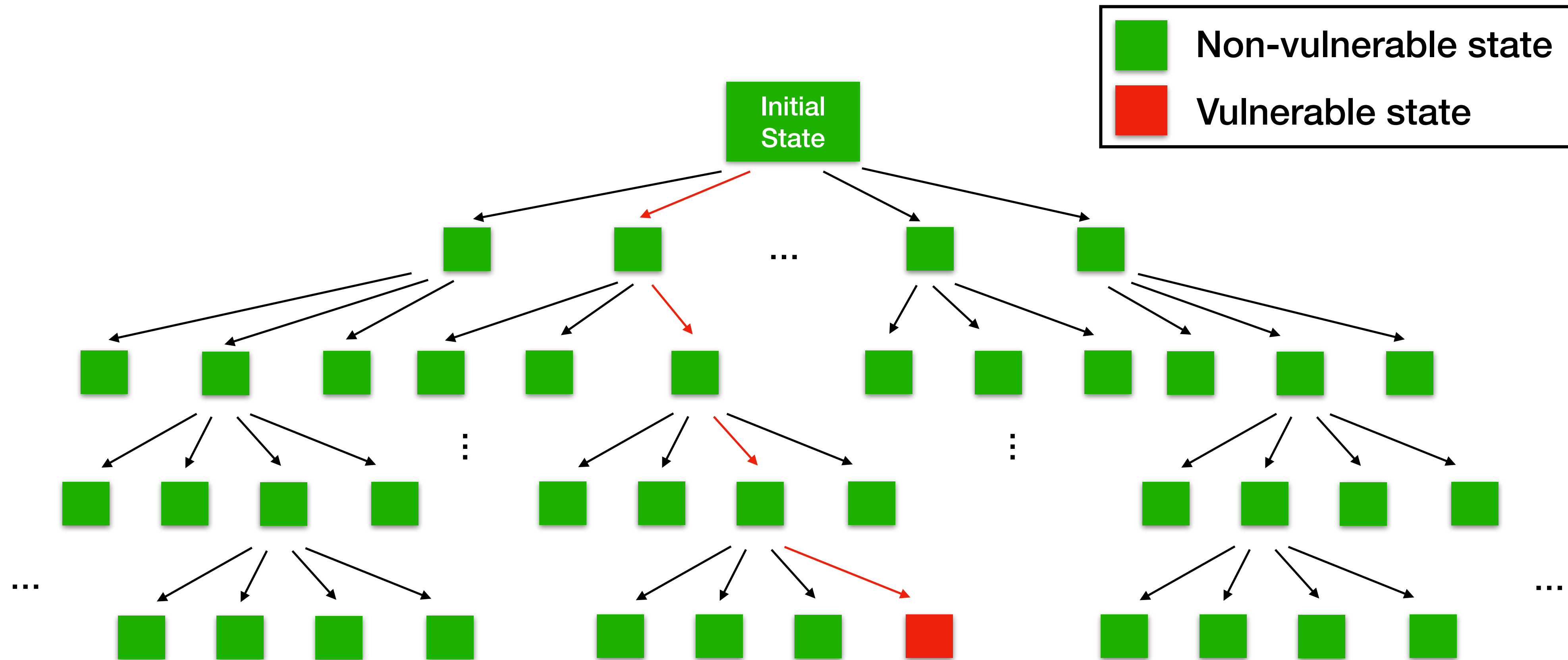
1. Enumerate the shortest sequence $t_0 \cdot t_1 \cdots t_n$.
2. Symbolically validate $t_0 \cdot t_1 \cdots t_n$.



3. Repeat Step 1, 2 until time limit expires.

Challenge: Path Explosion Problem

- **Huge search space** for transaction sequences



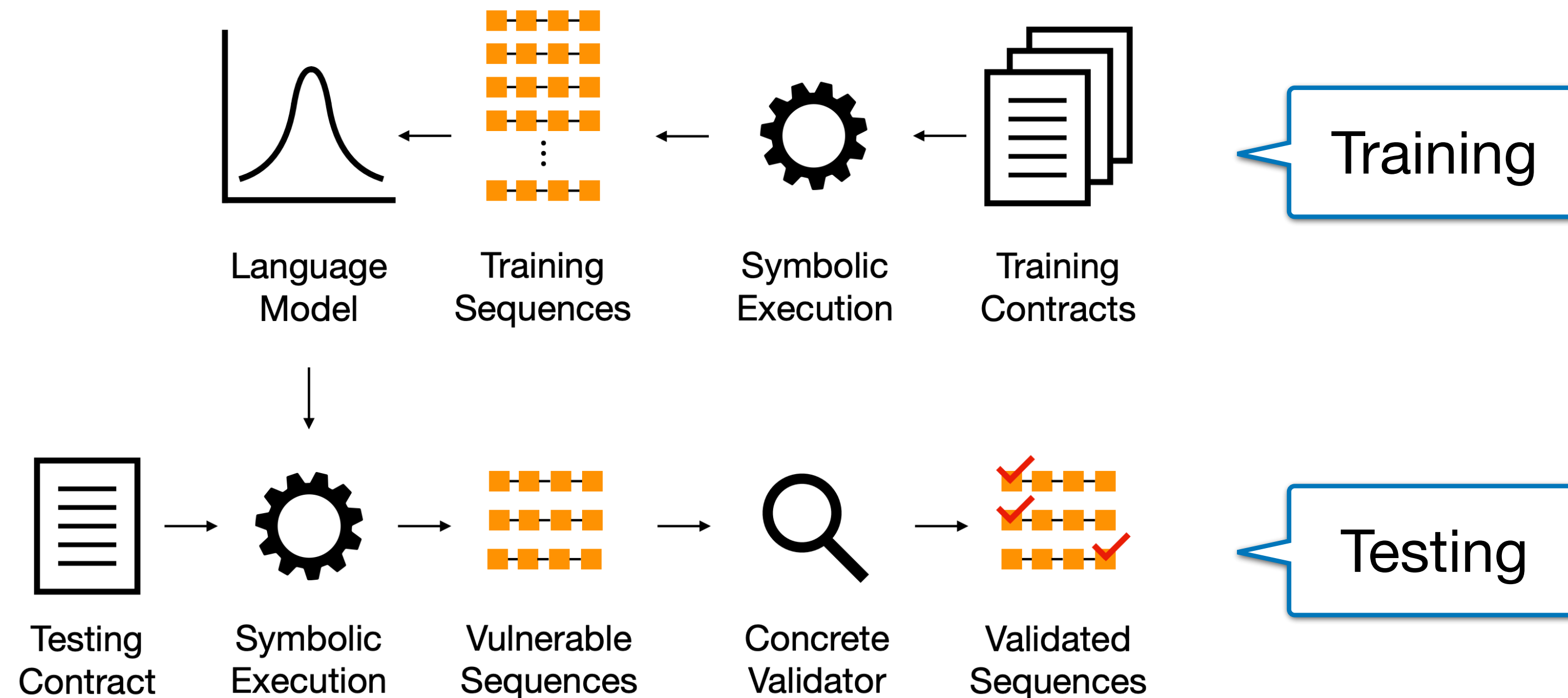
Key Idea:

Language Model-Guided Symbolic Execution

- Prioritize transaction sequences likely to contain vulnerabilities.

mintToken → approve → transferFrom → ... **70%**

transferFrom → approve → approve → ... **0.2%**



Learning a Language Model

- Goal: construct a training corpus (word sequences for computing n-gram counts).
- Challenge: how to abstract transaction sequences for effective generalization.
- Our Idea: use **type information**.
 - Observation: each function contains certain types of variables only!

```
contract ERC20 {  
    address owner;  
    mapping (address => uint) balance;  
    mapping (address => mapping (address => uint)) allowed;  
    uint totalSupply;  
  
    ...  
  
    function setOwner (address newOwner) {  
        require (owner == msg.sender);  
        owner = newOwner;  
    }  
}
```

No variables other than
address types

Learning a Language Model

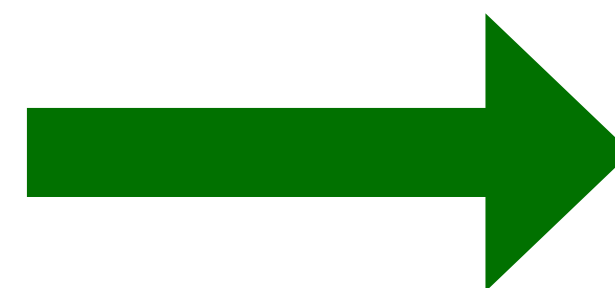
1. Collect vulnerable training sequences $\{T_1, \dots, T_n\}$ from training contracts.
2. Construct a **type frequency table** by counting the type frequencies from $\{T_1, \dots, T_n\}$.

Type	mapping (address=>uint)	uint	address
Frequency	2,100	1,400	1,200

3. Convert the training sequences into word sequences.

Transaction

```
function setOwner (address newOwner) {  
  require (owner == msg.sender);  
  owner = newOwner;  
}
```



Word

abstraction using
type frequency table

$\langle 0, 0, \mathbf{1}, 0, 0, \mathbf{1}, \dots \rangle$
 Def Use

Using a Language Model

- Prioritize likely sequences, according to the learned language model.

Transaction Sequence

$t_0 \cdot t_1 \cdot t_2$

abstraction using
type frequency table

Word Sequence

$\langle s \rangle \cdot \langle s \rangle \cdot w_0 \cdot w_1 \cdot w_2$

Pseudo-start word

Probability of
Being Vulnerable

$$\begin{aligned} &P(w_0 \mid s \cdot s) \times P(w_1 \mid s \cdot w_0) \times P(w_2 \mid w_0 \cdot w_1) \\ &= \frac{C(s \cdot s \cdot w_0)}{C(s \cdot s)} \times \frac{C(s \cdot w_0 \cdot w_1)}{C(s \cdot w_0)} \times \frac{C(w_0 \cdot w_1 \cdot w_2)}{C(w_0 \cdot w_1)} \end{aligned}$$

Evaluation: SmarTest vs. 5 Tools

Tool	Integer Overflow	Division by Zero	Assertion Violation	ERC20 Standard Violation	CVE Vulnerability Detection (Sample: 242)
SmarTest	1982	203	77	654	219
Mythril (ConsenSys)	460	73	25	n/a	85
Manticore (Trail of Bits)	2	1	3	n/a	0

90.5%

35.1%

Table 1. Results on CVE dataset. Found and validated vulnerable sequences.

Tool	Ether-leaking (90 contracts)			Suicidal (53 contracts)		
	#Contract	#Function	#Line	#Contract	#Function	#Line
SmarTest	81	111	111	51	51	51
ILF [CCS '19]	75	101	n/a	50	50	n/a
Maian [ACSAC '18]	58	n/a	n/a	43	n/a	n/a
teEther [USENIX Security '18]	37	n/a	n/a	n/a	n/a	n/a
Mythril (ConsenSys)	7	8	8	19	19	19
Manticore (Trail of Bits)	9	9	9	3	3	3

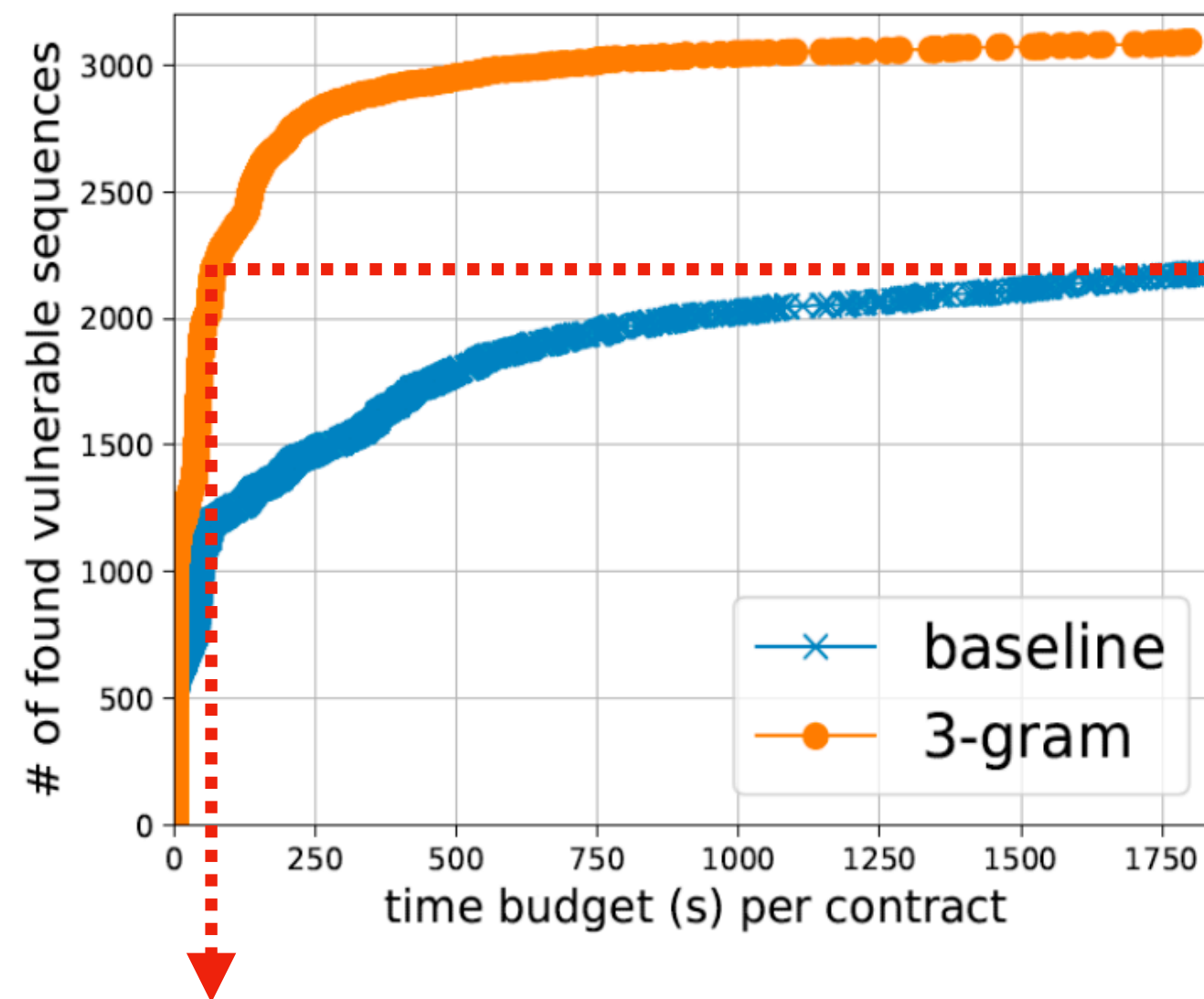
Table 2. Results on Leaking-Suicidal dataset. Found and validated (when available) vulnerable sequences.

Evaluation: Impact of Using Language Model

- Baseline: SmarTest without language model
- 3-gram: SmarTest with 3-gram language model

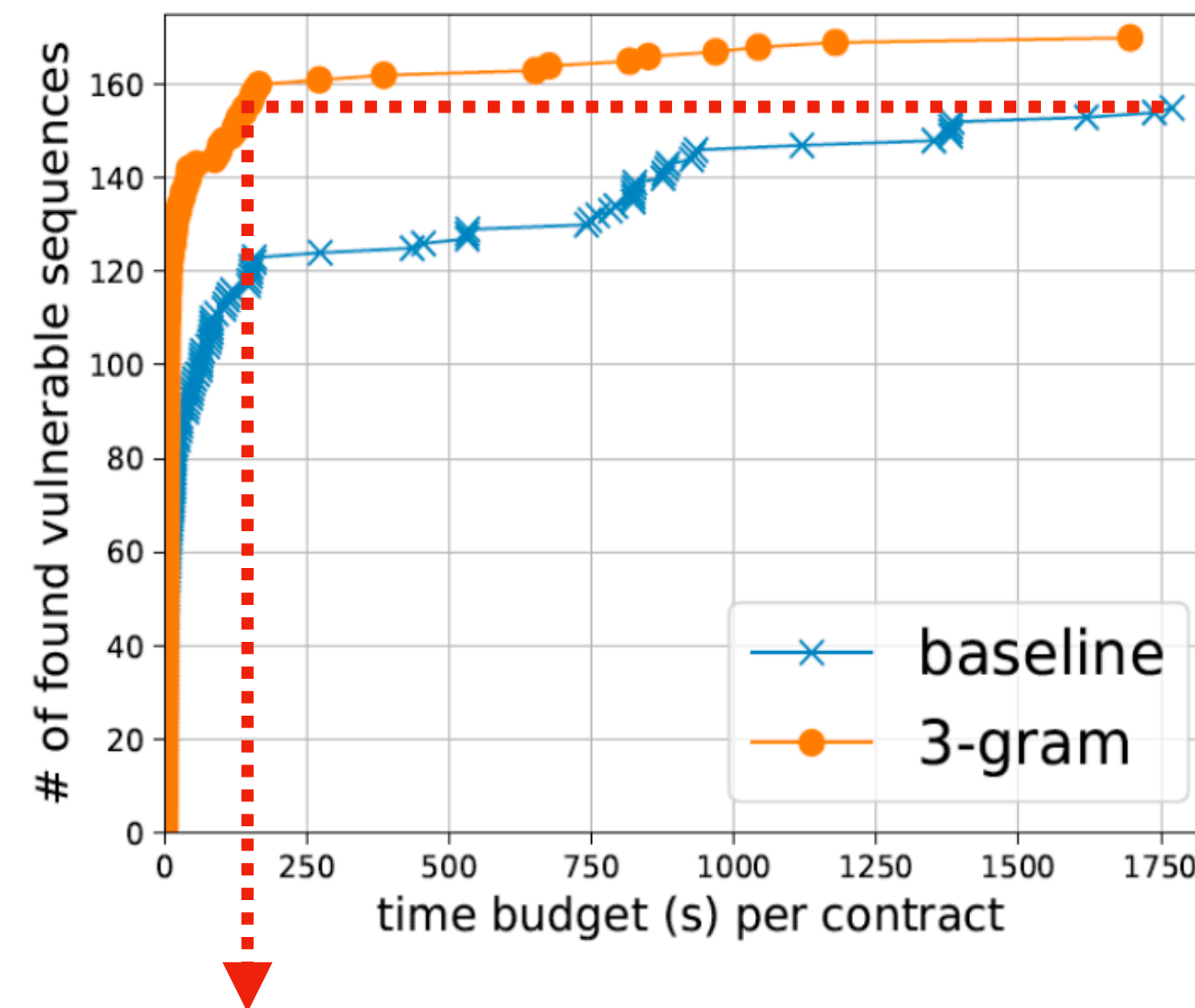
Speed-up
12-30 times

CVE dataset



68s (vs. 1,817s)

Leaking-Suicidal dataset



140s (vs. 1,770s)

Evaluation: Finding Zero-Day Vulnerabilities

- Ran SmarTest on 2,743 contracts from Etherscan.
- Manually confirmed **7 critical vulnerabilities** (ERC20 Standard Violation).

Pattern 1: Due to mistakenly named constructors, anyone can tokens for free.

```
1  contract AToken {
2      /* Constructor function */
3      function BToken () public {
4          balance[msg.sender] = 10000000000;
5          totalSupply = 10000000000;
6      }
7      ...
```

“BToken” at line 3 should be changed to “AToken”.

Pattern 2: Unrestricted token transfer by unauthorized users.

```
1  function transferFrom (address from, address to,
2      uint value) public returns (bool) {
3      require(balance[from] >= value);
4      require(balance[to] + value >= balance[to]);
5      require(allowed[from][msg.sender] >= value);
6      balance[from] -= value;
7      balance[to] += value;
8      return true;
9  }
```

Red-colored part is absent in the original code.

In Progress

Enhancing Symbolic Execution by Pruning Redundant Transaction Sequences

```
1  contract Example {
2    bool flag;
3    uint x;
4
5    constructor () public {
6      flag = false;
7      x = 0;
8    }
9
10   function setFlag (bool b) public {
11     flag = b;
12   }
13
14   function setX (uint y) public {
15     x = y;
16   }
17
18   function setXto10 () public {
19     x = 10;
20   }
21
22   function hasError () public {
23     require (flag == true);
24     require (x > 5);
25     assert (false);
26   }
27 }
```

Goal:
Trigger the error

In Progress

Enhancing Symbolic Execution by Pruning Redundant Transaction Sequences

```
1  contract Example {
2    bool flag;
3    uint x;
4
5    constructor () public {
6      flag = false;
7      x = 0;
8    }
9
10   function setFlag (bool b) public {
11     flag = b;
12   }
13
14   function setX (uint y) public {
15     x = y;
16   }
17
18   function setXto10 () public {
19     x = 10;
20   }
21
22   function hasError () public {
23     require (flag == true);
24     require (x > 5);
25     assert (false);
26   }
27 }
```

Constructor ()

TSEQ1

setX ()

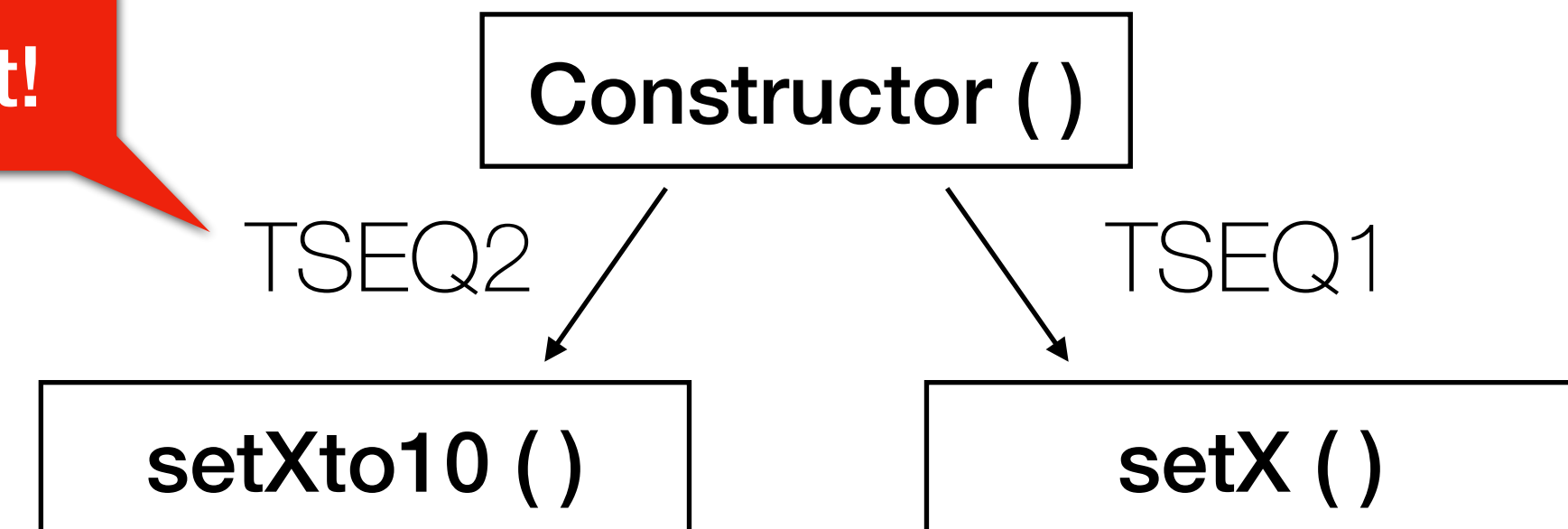
Goal:
Trigger the error

In Progress

Enhancing Symbolic Execution by Pruning Redundant Transaction Sequences

```
1  contract Example {
2    bool flag;
3    uint x;
4
5    constructor () public {
6      flag = false;
7      x = 0;
8    }
9
10   function setFlag (bool b) public {
11     flag = b;
12   }
13
14   function setX (uint y) public {
15     x = y;
16   }
17
18   function setXto10 () public {
19     x = 10;
20   }
21
22   function hasError () public {
23     require (flag == true);
24     require (x > 5);
25     assert (false);
26   }
27 }
```

Redundant!



Goal:
Trigger the error

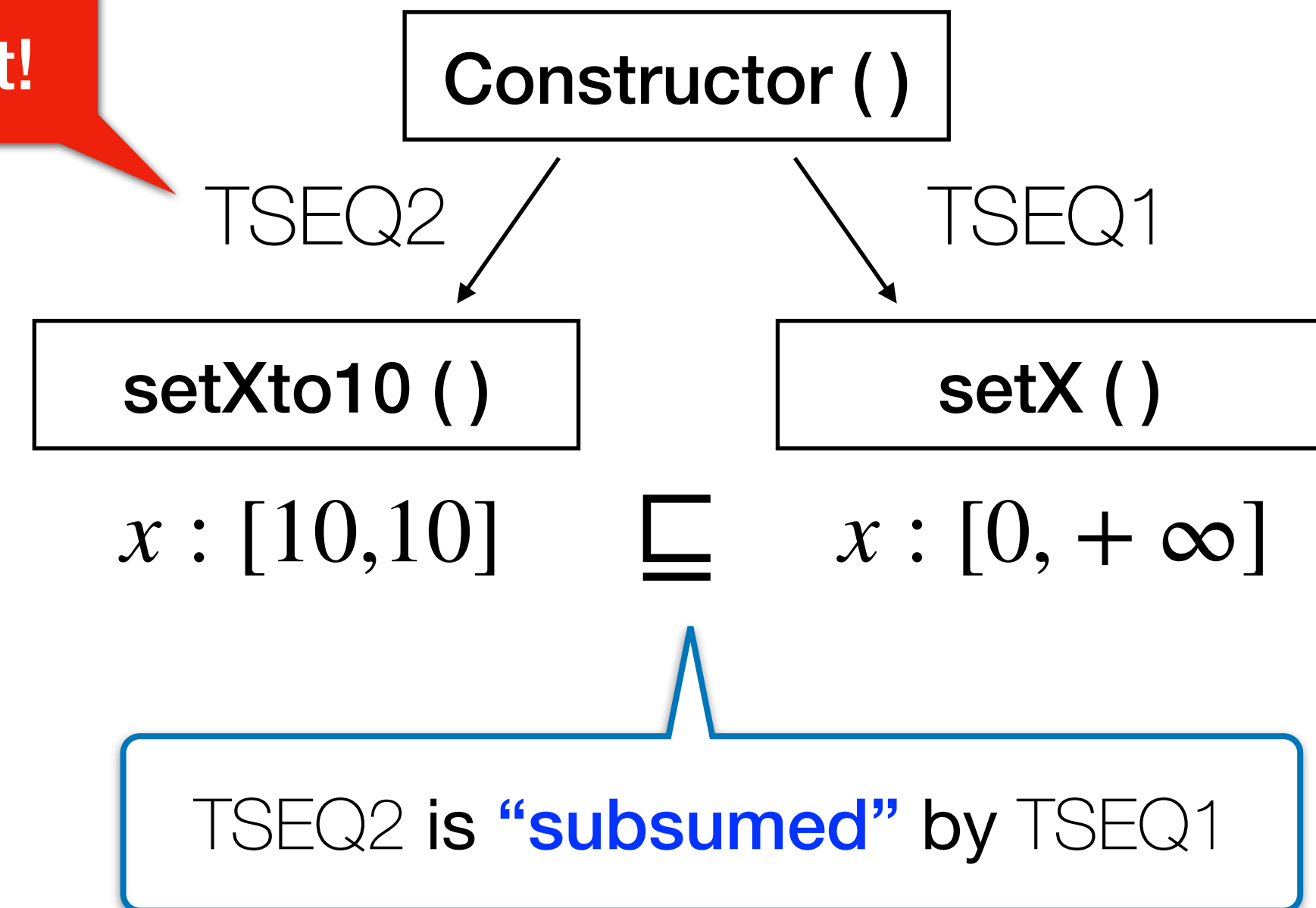
Enhancing Symbolic Execution by Pruning Redundant Transaction Sequences

```

1  contract Example {
2    bool flag;
3    uint x;
4
5    constructor () public {
6      flag = false;
7      x = 0;
8    }
9
10   function setFlag (bool b) public {
11     flag = b;
12   }
13
14   function setX (uint y) public {
15     x = y;
16   }
17
18   function setXto10 () public {
19     x = 10;
20   }
21
22   function hasError () public {
23     require (flag == true);
24     require (x > 5);
25     assert (false);
26   }
27 }

```

Redundant!



Goal:
Trigger the error

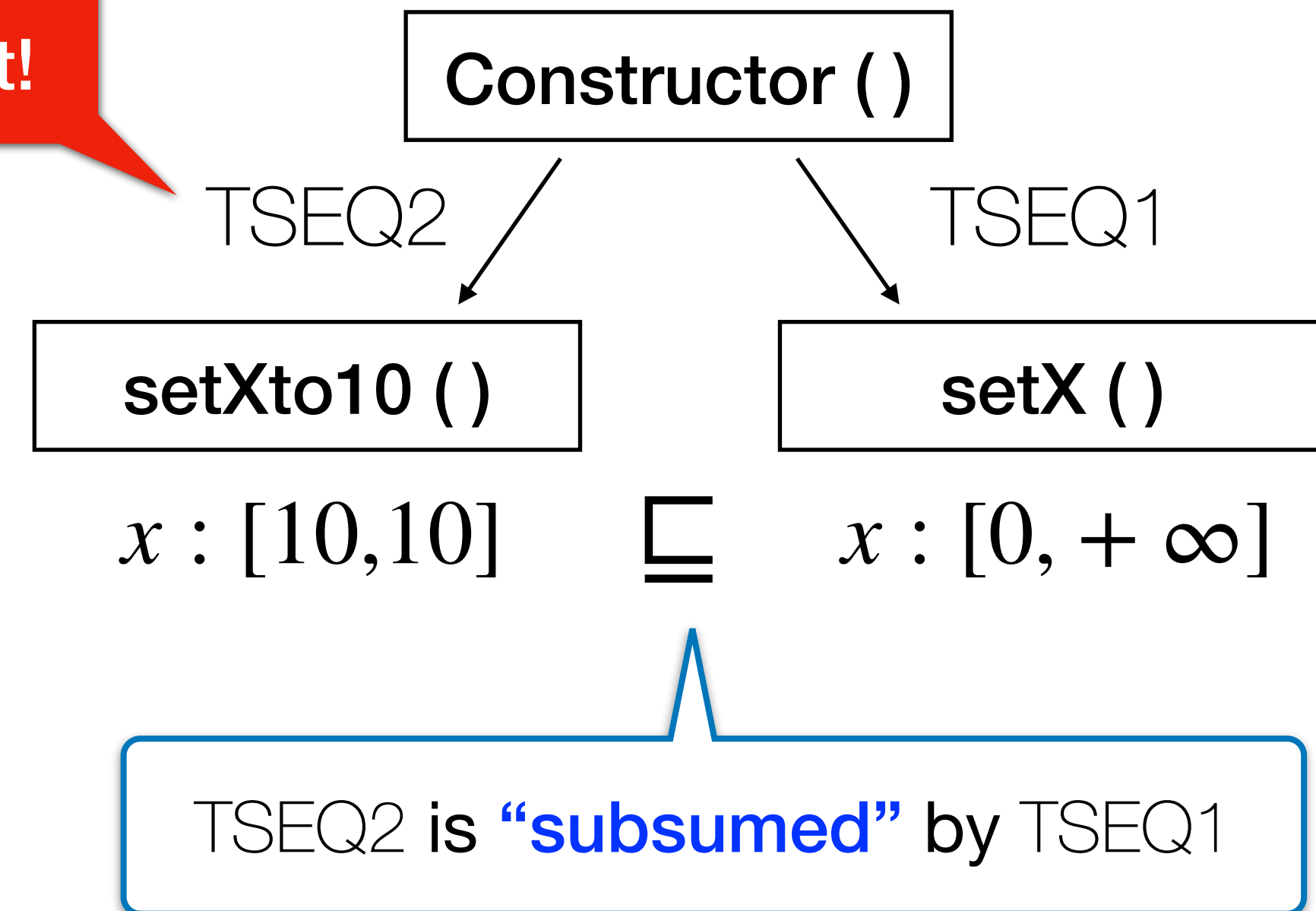
Enhancing Symbolic Execution by Pruning Redundant Transaction Sequences

```

1  contract Example {
2    bool flag;
3    uint x;
4
5    constructor () public {
6      flag = false;
7      x = 0;
8    }
9
10   function setFlag (bool b) public {
11     flag = b;
12   }
13
14   function setX (uint y) public {
15     x = y;
16   }
17
18   function setXto10 () public {
19
20
21
22
23
24
25
26
27 }

```

Redundant!



- **No existing approaches** can soundly prune redundant sequences!
- **Our solution**: design a static analysis to identify the redundancy



Enhancing Symbolic Execution by Pruning Redundant Transaction Sequences

- Initial results: succeeded in improving the vulnerability-finding capability of baseline

Contract Name	Baseline (increasing order)	Ours (Baseline + Pruning)
cve-2018-10376.sol	4	5 (+1)
cve-2018-11687.sol	3	4 (+1)
cve-2018-13161.sol	7	7
cve-2018-13570.sol	1	2 (+1)
cve-2018-13700.sol	7	8 (+1)
cve-2018-13701.sol	4	7 (+3)
cve-2018-14004.sol	7	8 (+1)
cve-2018-14084.sol	9	10 (+1)
cve-2018-14087.sol	2	5 (+3)
cve-2018-17050.sol	3	3
Total (#found vul.)	47	59 (+12)

3. SmartFix: Verification-based Repair

*SmartFix: Fixing Vulnerable Smart Contracts by Accelerating
Generate-and-Verify Repair using Statistical Models*
FSE 2023

SmartFix's Goal:

Fixing Vulnerable Contracts Automatically and Safely

CVE-2018-11411

```
1  function transferFrom (address from, address to, uint value) returns (bool success) {
2      if (value == 0) return false;
3      uint fromBalance = balance[from];
4      uint allowance = allowed[from][msg.sender];
5
6      bool sufficientFunds = fromBalance <= value;
7      bool sufficientAllowance = allowance <= value;
8      bool overflowed = balance[to] + value > balance[to];
9
10     if(sufficientFunds && sufficientAllowance && !overflowed) {
11         balance[to] += value;           // overflow
12         balance[from] -= value;         // underflow
13         allowed[from][msg.sender] -= value; // underflow
14         return true;
15     }
16     else {return false;}
17 }
```

SmartFix's Goal:

Fixing Vulnerable Contracts Automatically and Safely

CVE-2018-11411

Our Goal

Line 6 : Replace `<=` by `>=`
Line 7 : Replace `<=` by `>=`
Line 8 : Replace `>` by `<`

```
1  function transferFrom (address from, address to, uint value) returns (bool success) {
2      if (value == 0) return false;
3      uint fromBalance = balance[from];
4      uint allowance = allowed[from][msg.sender];
5
6      bool sufficientFunds = fromBalance <= value;
7      bool sufficientAllowance = allowance <= value;
8      bool overflowed = balance[to] + value > balance[to];
9
10     if(sufficientFunds && sufficientAllowance && !overflowed) {
11         balance[to] += value;           // overflow
12         balance[from] -= value;         // underflow
13         allowed[from][msg.sender] -= value; // underflow
14         return true;
15     }
16     else {return false;}
17 }
```

Limitations of Existing Techniques:

Simple Patch Only

Rely on a single repair template for each bug type

CVE-2018-11411

```
1  function transferFrom (address from, address to, uint value) returns (bool success) {
2      if (value == 0) return false;
3      uint fromBalance = balance[from];
4      uint allowance = allowed[from][msg.sender];
5
6      bool sufficientFunds = fromBalance <= value;
7      bool sufficientAllowance = allowance <= value;
8      bool overflowed = safeAdd(balance[to],value) > balance[to];
9
10     if(sufficientFunds && sufficientAllowance && !overflowed) {
11         balance[to] = safeAdd(balance[to],value);
12         balance[from] = safeSub(balance[from],value);
13         allowed[from][msg.sender] = safeSub(allowed[from][msg.sender],value);
14         return true;
15     }
16     else {return false;}
17 }
```

sGuard [IEEE S&P'21], SmartShield [SANER'20], Elysium [RAID'22]:
Rely only on inserting runtime checks

✱ safeAdd/safeSub: raise exceptions if over/underflows occur at runtime

Limitations of Existing Techniques:

Simple Patch Only

Rely on a single repair template for each bug type

CVE-2018-11411

```
1  function transferFrom (address from, address to, uint value) returns (bool success) {
2      if (value == 0) return false;
3      uint fromBalance = balance[from];
4      uint allowance = allowed[from][msg.sender];
5
6      bool sufficientFunds = fromBalance <= value;
7      bool sufficientAllowance = allowance <= value;
8      bool overflowed = safeAdd(balance[to],value) > balance[to];
9
10     if(sufficientFunds && sufficientAllowance && !overflowed) {
11         balance[to] = safeAdd(balance[to],value);
12         allowance[from][msg.sender] = safeSub(allowance[from][msg.sender],value);
13         return true;
14     }
15     else {return false;}
16 }
17 }
```

To pass line 8:
A: balance[to] + value >= balance[to]

sGuard [IEEE S&P'21], SmartShield [SANER'20], Elysium [RAID'22]:
Rely only on inserting runtime checks

✱ safeAdd/safeSub: raise exceptions if over/underflows occur at runtime

Limitations of Existing Techniques:

Simple Patch Only

Rely on a single repair template for each bug type

CVE-2018-11411

```
1  function transferFrom (address from, address to, uint value) returns (bool success) {
2      if (value == 0) return false;
3      uint fromBalance = balance[from];
4      uint allowance = allowed[from][msg.sender];
5
6      bool sufficientFunds = fromBalance <= value;
7      bool sufficientAllowance = allowance <= value;
8      bool overflowed = safeAdd(balance[to], value) > balance[to];
9
10     if(sufficientFunds && sufficientAllowance && !overflowed) {
11         balance[to] = safeAdd(balance[to], value);
12         allowance[from] = safeSub(allowance[from], value);
13         return true;
14     }
15     else {return false;}
16 }
17 }
```

To pass line 8:
A: $\text{balance}[\text{to}] + \text{value} \geq \text{balance}[\text{to}]$

B: $\text{balance}[\text{to}] + \text{value} \leq \text{balance}[\text{to}]$

sGuard [IEEE S&P'21], SmartShield [SANER'20], Elysium [RAID'22]:
Rely only on inserting runtime checks

✱ safeAdd/safeSub: raise exceptions if over/underflows occur at runtime

Limitations of Existing Techniques:

Simple Patch Only

Rely on a single repair template for each bug type

CVE-2018-11411

```
1  function transferFrom (address from, address to, uint value) returns (bool success) {
2      if (value == 0) return false;
3      uint fromBalance = balance[from];
4      uint allowance = allowed[from][msg.sender];
5
6      bool sufficientFunds = fromBalance <= value;
7      bool sufficientAllowance = allowance <= value;
8      bool overflowed = safeAdd(balance[to],value) > balance[to];
9
10     if(sufficientFunds && sufficientAllowance && !overflowed) {
11         balance[to] = safeAdd(balance[to],value);
12         allowance[from] = safeSub(allowance[from],value);
13         return true;
14     }
15     else {return false;}
16 }
17 }
```

To pass line 8:
A: $\text{balance}[\text{to}] + \text{value} \geq \text{balance}[\text{to}]$

B: $\text{balance}[\text{to}] + \text{value} \leq \text{balance}[\text{to}]$

sGuard [IEEE S&P'21], SmartShield [SANER'20], Elysium [RAID'22]:
Rely only on inserting runtime checks

✱ safeAdd/safeSub: raise exceptions if over/underflows occur at runtime

Limitations of Existing Techniques:

Simple Patch Only

Rely on a single repair template for each bug type

CVE-2018-11411

```
1 function transferFrom (address from, address to, uint value) returns (bool success) {
2   if (value == 0) return false;
3   uint fromBalance = balance[from];
4   uint allowance = allowed[from][msg.sender];
5
6   bool sufficientFunds = fromBalance <= value;
7   bool sufficientAllowance = allowance <= value;
8   bool overflowed = safeAdd(balance[to], value) > balance[to];
9
10  if(sufficientFunds && sufficientAllowance && !overflowed) {
11    balance[to] = safeAdd(balance[to], value);
12    allowance[from] = safeSub(allowance[from], value);
13    return true;
14  }
15  else {return false;}
16 }
17 }
```

value != 0 after line 2

value == 0 in if-branch (by A,B)

To pass line 8:
A: balance[to] + value >= balance[to]
B: balance[to] + value <= balance[to]

sGuard [IEEE S&P'21], SmartShield [SANER'20], Elysium [RAID'22]:
Rely only on inserting runtime checks

✱ safeAdd/safeSub: raise exceptions if over/underflows occur at runtime

Limitations of Existing Techniques:

Simple Patch Only

Rely on a single repair template for each bug type

CVE-2018-11411

```
1 function transferFrom (address from, address to, uint value) returns (bool) {
2   if (value == 0) return false;
3   uint fromBalance = balance[from];
4   uint allowance = allowed[from][msg.sender];
5
6   bool sufficientFunds = fromBalance <= value;
7   bool sufficientAllowance = allowance <= value;
8   bool overflowed = safeAdd(balance[to], value) > balance[to];
9
10  if(sufficientFunds && sufficientAllowance && !overflowed) {
11    balance[to] = safeAdd(balance[to], value);
12    allowance[from][msg.sender] -= value;
13    return true;
14  }
15  else {return false;}
16 }
17 }
```

value!=0 after line 2

Incorrect Patch (Deadcode)

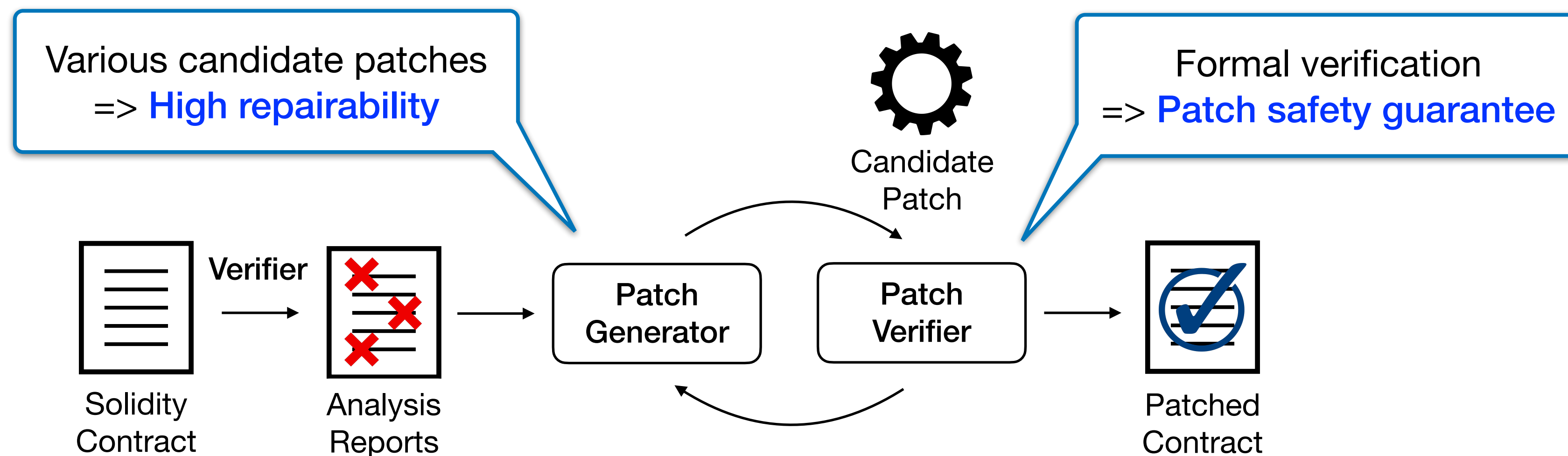
value == 0 in if-branch (by A,B)

To pass line 8:
A: balance[to] + value >= balance[to]
B: balance[to] + value <= balance[to]

sGuard [IEEE S&P'21], SmartShield [SANER'20], Elysium [RAID'22]:
Rely only on inserting runtime checks

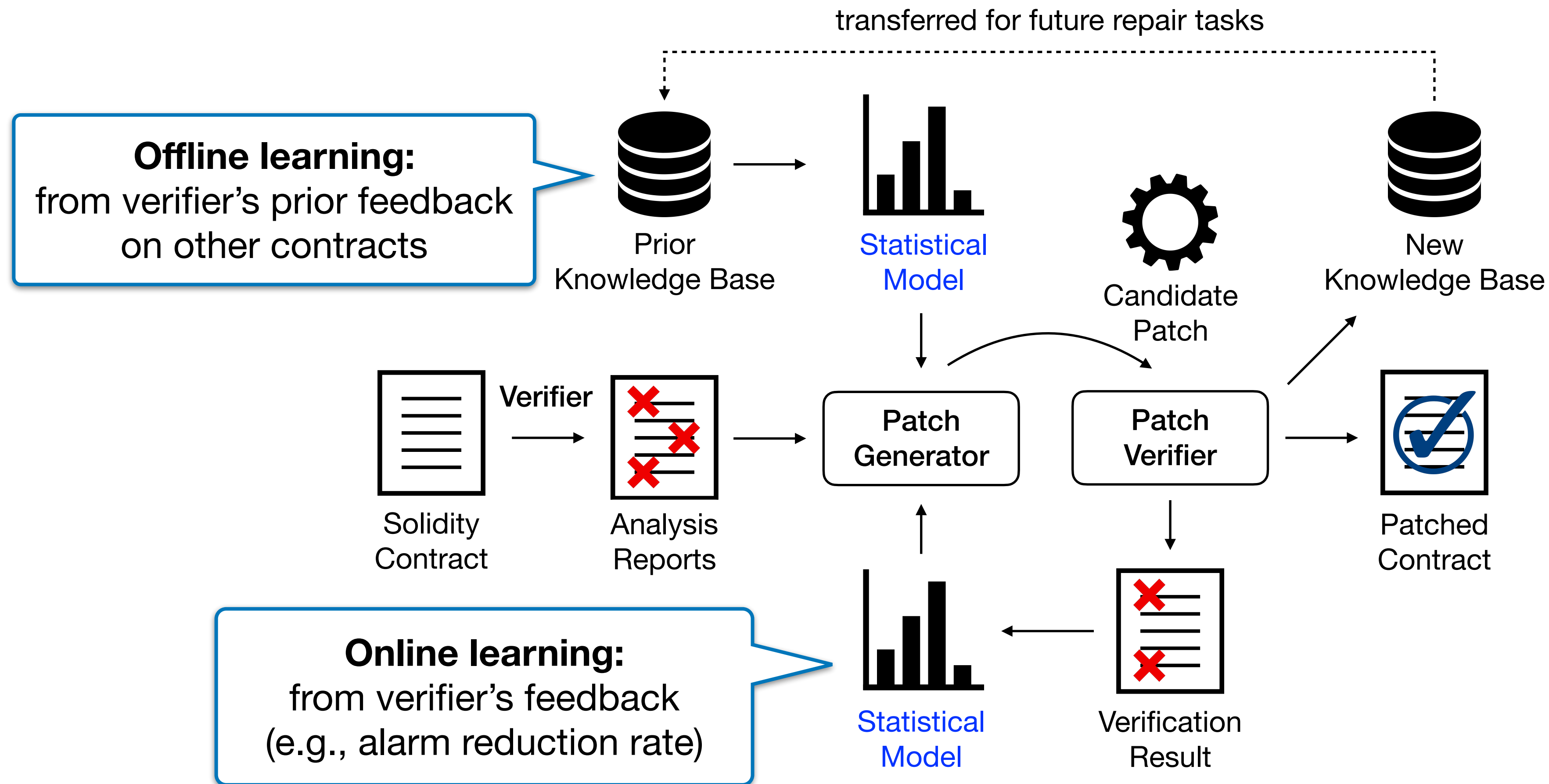
✱ safeAdd/safeSub: raise exceptions if over/underflows occur at runtime

Basic Approach: Generate-and-Verify



Challenge: Repair efficiency
(large search space + patch verification cost)

SmartFix Approach: Speeding up Generate-and-Verify using Statistical Models



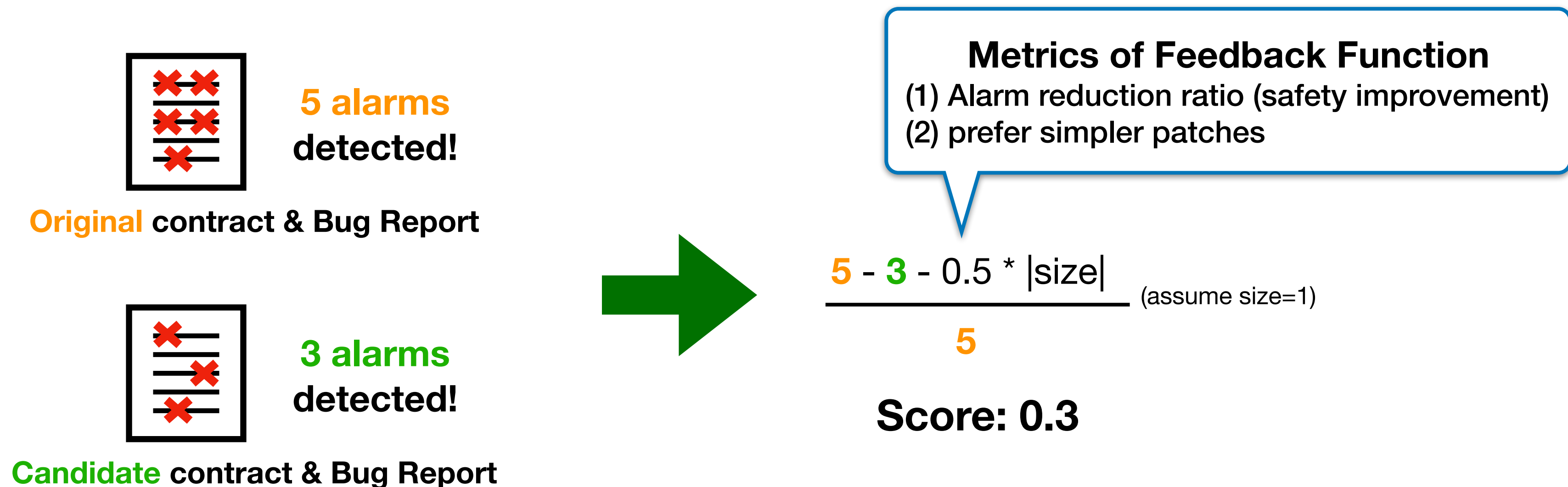
Key Idea: prioritize likely candidates using learned models

Learning a Model M_{on} Online

1. Verify a candidate patch p .
2. Generate a training data $d = (feature, score)$
 - *feature*: vector representation of p
 - *score*: a real number that quantifies the verification result for p
3. Repeat Steps 1 and 2, and collect a training dataset D .
4. Learn M_{on} from D using an off-the-shelf supervised learning algorithm for regression.

Learning a Model M_{on} Online

- Feature representation: inclusion of each patch component
 - E.g., a patch contains the 1st and the 3rd patch components $\longrightarrow \langle 1, 0, 1, 0, 0 \rangle$
- Score: compute alarm reduction rate w.r.t. the original contract

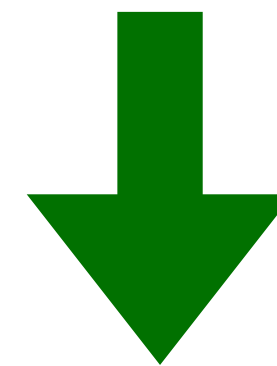


Learning a Model M_{off} Offline

1. Construct a knowledge base $K = \{(\hat{p}_1, score_1), \dots, (\hat{p}_n, score_n)\}$ from past repairs.
 - $\hat{p}$: abstract patch without contract-specific information (e.g., line number)

Patch
(line-level information)

```
Insert(2, require(totalSupply + amount >= totalSupply);
```



Type Information

Abstract Patch
(function-level feature)

```
Abs_Insert(<{mapping, uint}, {address}, ...>, {uint})
```

Feature of a function that contains line 2.

Feature of a patch expression/statement

Learning a Model M_{off} Offline

1. Construct a knowledge base $K = \{(\hat{p}_1, score_1), \dots, (\hat{p}_n, score_n)\}$ from past repairs.
 - $\hat{p}$: abstract patch without contract-specific information (e.g., line number)
2. Obtain $D_{off} = \{(vector_1, score_1), \dots, (vector_n, score_n)\}$ from $K = \{(\hat{p}_1, score_1), \dots, (\hat{p}_n, score_n)\}$.
 - based on the inclusion of each abstract patch component
 - E.g., $\hat{p}$ contains the 1st and the 3rd abstract patch components $\longrightarrow <1,0,1,0,0>$
3. Learn M_{off} model using an off-the-shelf supervised learning algorithm for regression.

Using Learned Models (M_{on} , M_{off})

- If M_{off} is learned, use M_{off}
- If M_{off} is not learned, use M_{on}
- If M_{off} is not useful, use M_{on}
 - a solution patch has not been generated within a predetermined loop bound
 - obtained a negative feedback (i.e., $score < 0$)

Evaluation: Setup

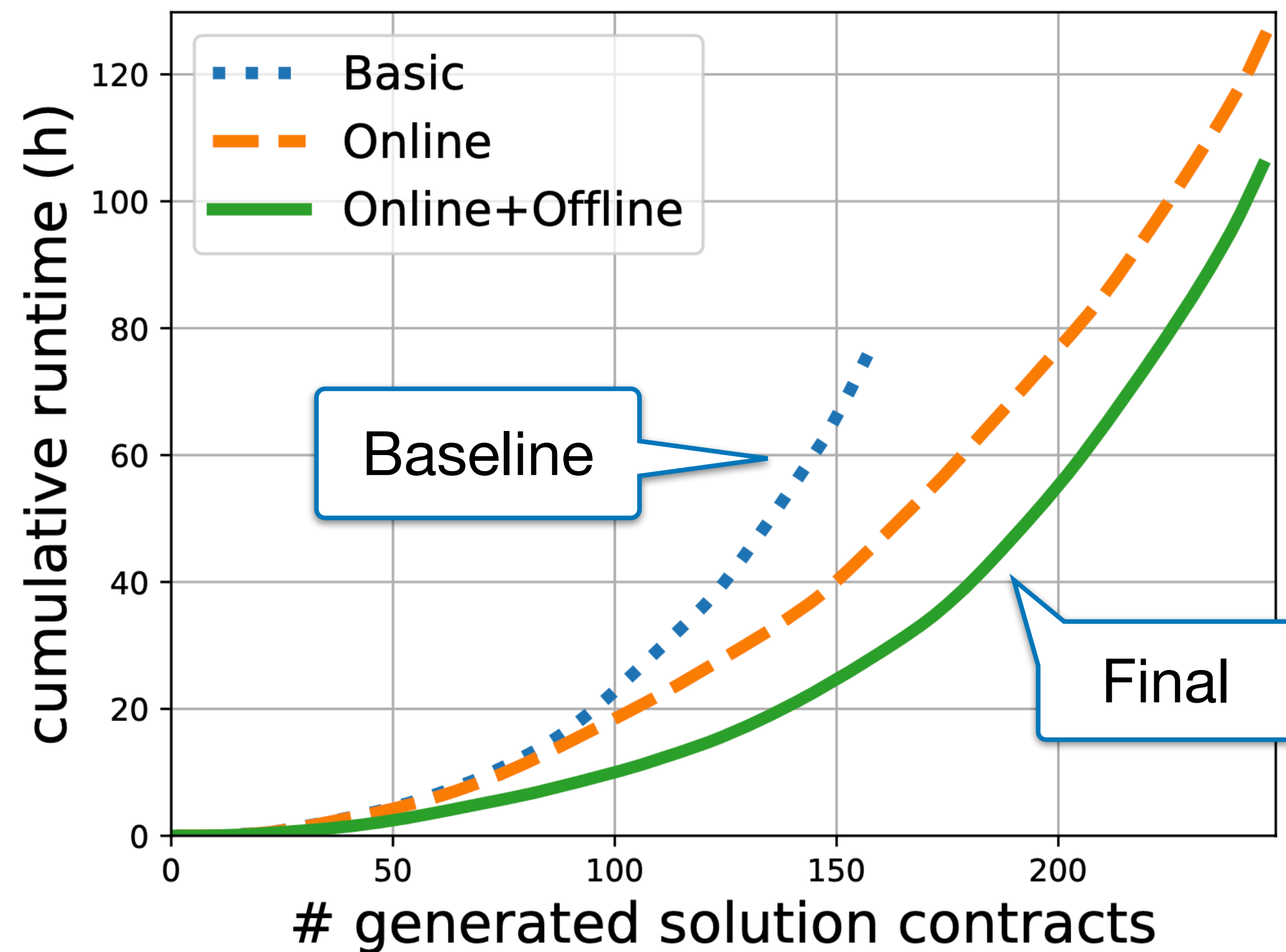
- **Comparison Target:** sGuard [IEEE S&P'21]
 - State-of-the-art fixing tool for Solidity smart contracts.
- **Benchmark:** collected 361 contracts from multiple sources
 - Integer over/underflow: 200 CVE-reported contracts
 - Ether-Leak & Suicidal: 104 from SmartTest [USENIX Sec'21]
 - Reentrancy: 28 from (SODA [NDSS'20], SmartBugs [ICSE'20]) + 17 by bug-injection + 2 from the wild
 - Dangerous tx.origin: 1 from SmartBugs [ICSE'20] + 9 by bug-injection

Evaluation: Fix Rate (vs. sGuard [IEEE S&P'21])

Bug Type	#Bug	SmartFix					sGuard [IEEE S&P '21]				
		#BugRun	#Generated	#Correct	Fix Succ. Rate	Accuracy	#BugRun	#Generated	#Correct	Fix Succ. Rate	Accuracy
IO	229	228	218	218	95.6%	100.0%	170	103	103	60.6%	100.0%
RE	52	51	46	46	90.2%	100.0%	33	33	29	87.9%	87.9%
TX	12	12	12	12	100.0%	100.0%	2	2	2	100.0%	100.0%
EL	137	134	83	76	56.7%	91.6%	n/a	n/a	n/a	n/a	n/a
SU	53	51	40	34	66.7%	85.0%	n/a	n/a	n/a	n/a	n/a
IO+RE+TX	293	291	276	276	94.8%	100.0%	205	138	134	65.4%	97.1%
Total	483	476	399	386	81.1%	96.7%	-	-	-	-	-

Fix Success Rate:
94.8% (Ours) vs. **65.4% (sGuard)**

Evaluation: Impact of Using Statistical Models



- #Generated Bug-free contracts
 - Baseline (157) vs. Final (246)

**Performance Up
56.7%**

$$= \frac{246 - 157}{157}$$

Q. Will LLMs Replace Formal Methods?

Case Study (1): Limitations of LLM

- LLMs are not as precise as formal verification.

아래 솔리디티 0.4.X 버전 컨트랙트에서, 정수 오버플로우/정수 언더플로우 취약점 전체 리스트를 라인별로 정확히 알려줘. 핵심만 간결하게. 문제가 없으면 없다고 알려주고. 답변 처음에 "라인 X,Y,Z 에 취약점이 존재합니다" 와 같은 형식으로 결론부터 두괄식으로 요약해줘.

```
1 contract Netkoin {  
2   mapping (address => uint) public balance;  
3   uint public totalSupply;  
4  
5   constructor (uint initialSupply) {
```

Prompt provided to
ChatGPT (v.4o),
Gemini (v.1.5 Flash)

Case Study (1): Limitations of LLM

- LLMs are not as precise as formal verification.

Formally proven to be safe by VeriSmart

False alarm by
ChatGPT (v.4o),
Gemini (v.1.5 Flash)

False alarm by
ChatGPT (v.4o),
Gemini (v.1.5 Flash)

```
1  contract Netkoin {
2      mapping (address => uint) public balance;
3      uint public totalSupply;
4
5      constructor (uint initialSupply) {
6          totalSupply = initialSupply;
7          balance[msg.sender] = totalSupply;
8      }
9
10     function transfer (address to, uint value) public
11     returns (bool) {
12         require (balance[msg.sender] >= value);
13         balance[msg.sender] -= value;
14         balance[to] += value;
15         return true;
16     }
17
18     function burn(uint value) public returns (bool) {
19         require (balance[msg.sender] >= value);
20         balance[msg.sender] -= value;
21         totalSupply -= value;
22         return true;
23     }
24 }
```

False alarm by ChatGPT (v.4o)

False alarm by ChatGPT (v.4o)

Case Study (2): Limitations of LLM

- LLMs may miss critical bugs.

Prompt provided to
Gemini (v.1.5 Flash)

S

아래 솔리디티 0.4.X 버전 함수에서, 정수 오버플로우/정수 언더플로우 취약점 전체 리스트를 라인 별로 정확히 알려줘. 핵심만 간결하게. 문제가 없으면 없다고 알려주고. 답변 처음에 "라인 X,Y,Z 에 취약점이 존재합니다" 와 같은 형식으로 결론부터 두괄식으로 요약해줘.¹

```
function transferProxy (address from, address to, uint value, uint fee)
```

```
2 public returns (bool) {
```

```
3   if (balance[from] < fee + value)
```

```
4     revert ();
```

```
5
```

Case Study (2): Limitations of LLM

- LLMs may miss critical bugs.

Detected by
VeriSmart, SmarTest

```
1  function transferProxy (address from, address to, uint value, uint fee)
2  public returns (bool) {
3      if (balance[from] < fee + value)
4          revert ();
5
6      if (balance[to] + value < balance[to] ||
7          balance[msg.sender] + fee < balance[msg.sender])
8          revert ();
9
10     balance[to] += value;
11     balance[msg.sender] += fee;
12     balance[from] -= value + fee;
13     return true;
14 }
```

Gemini (v.1.5 Flash) failed to detect
in all attempts (> 10)

CVE-2018-10376

Lessons from Case Studies

LLMs will **NOT replace formal methods!**

- Unlike formal methods, **LLMs cannot formally prove/disprove** SW security & safety.
 - due to their statistical nature — (can only produce “likely” answers).
- LLMs are not sufficient for analyzing safety-critical SW.

Summary:

Formal Methods for Smart Contract Security

- Existing approaches relied on lightweight, brittle analysis methods.
- **Our Rigorous Formal Methods**
 - VeriSmart: formal methods for safety verification
 - SmarTest: formal methods for effective vulnerability-finding
 - SmartFix: formal methods for fixing vulnerabilities

**The first successful stories of formal methods
for smart contract security**

Sourcode & Benchmark: <https://github.com/kupl/VeriSmart-public>